QSC Rust Wrapper – QSCRS

Technical Specification · Revision 0.2.0 · November 8, 2026

Quantum Resistant Cryptographic Solutions Corporation

This engineering specification defines the QSCRS source-level Rust API, its QSCABI 0.10 dynamic FFI boundary, safety and ownership models, installation and runtime requirements, cryptographic and transport components, tests, examples and public function inventory.

QSCRS is QRCS commercial software. Applicable license: QRCS Commercial Application Software License (QRCS-CASL). This paper does not alter the controlling software license or confer redistribution rights.

Contents

1 Foreword 3
2 Introduction and Source Baseline 3
3 References and Conventions 4
4 Architecture and Crate Organization 6
5 Build and Native Runtime Loading 7
6 Errors, Status and Ownership 8
7 Version and Frozen ABI Contract 9
8 Hash, Encoding and XOF APIs 10
9 MAC and Key Derivation 11
10 Symmetric Encryption and AEAD 12
11 Post-Quantum Key Encapsulation 12
12 Digital Signatures and Classical Curves 13
13 Explicit Algorithm Parameter APIs 14
14 Streaming and Stateful Cryptography 14
15 X.509 Certificates and Issuance 15
16 TLS 1.3 and Network Lifecycle 16
17 Hardware Provider and Key Custody 17
18 Rust Examples and Integration Patterns 17

19 Tests and Qualification Evidence 18
20 Deployment, Limitations and Security Review 19
21 Complete Public Rust API Reference 21
22 Low-Level qsc-sys Binding Reference 69
23 Implementor Acceptance Checklist 91
Annex A Public Rust Types and Selectors 92
Annex B Native ABI and Source Directory Index 93
Annex C Testing and Example Inventory 94

1: Foreword

QSCRS is the Rust-language wrapper for the QSC cryptographic SDK, implemented on the native QSCABI library. Its two principal crates, qsc and qsc-sys, deliberately separate safe application use from unsafe native calls. The user-provided QSCRS(2).zip also includes staged Rust tests, interactive examples, a dedicated test launcher, Windows integration scripts and a frozen native export contract.

This document is an implementor specification. It uses the source declarations as authority and distinguishes APIs actually present from supporting native symbols and from features that are optional, unavailable or dependent on external binaries. It does not assert that the Rust wrapper or native QSC library was compiled or executed in preparing this document.

The presentation is matched to the QRCS QSC engineering-document reference: letter-sized pages, one-inch margins, Segoe UI body text, Segoe UI Semibold headings, numbered contents, full paragraph-width tables, black text except URLs, and explicit call hierarchies and API declarations.

2: Introduction and Source Baseline

2.1 Project scope

QSCRS is not a pure Rust cryptographic implementation. All cryptographic operations are delegated through qsc-sys to the compiled QSCABI C interface, which in turn invokes QSC. The safe Rust crate presents slices, Vec<u8>, SecretBytes, scoped state handles and Result<T, QscError>; the low-level binding crate resolves QSCABI symbols and marshals native scalars, status enums and pointers.

2.2 Version and scope baseline

Source property	Uploaded-project finding
Rust crate version	qsc, qsc-sys, qsc_examples and qsc_tests each declare package version 0.2.0 in their Cargo.toml files.
Native ABI baseline	329 named QSCABI 0.10 exports are recorded in qsc-sys/src/abi010_public_symbols.txt and required by qsc/src/abi_contract.rs.
Low-level dispatch model	252 eager function pointers plus 77 additional direct/optional FFI signatures, recorded in the frozen signature JSON and contract script.
Public safe Rust catalogue	378 pub fn entries identified in the 27 module files, including macro-generated functions; the reference reproduces their source signatures.

Source property	Uploaded-project finding
Explicit low-level Rust helpers	177 public functions directly declared in qsc-sys/src outside its dynamic loader (in addition to its eager function-pointer table).
Distribution license	QRCS-CASL proprietary commercial license texts in source headers.
Workspace completeness	The archive has four member Cargo.toml manifests but no workspace-root Cargo.toml, although members inherit workspace metadata and the Visual Studio script invokes Cargo at the workspace root.
Build/test verification	This document reflects source inspection, not a newly executed Cargo build, Rust tests, or target-platform qualification.

Table 2.1 Supplied source baseline

2.3 Document interpretation

A returned Vec<u8> is caller-owned Rust memory. A SecretBytes value has a drop-time erasure policy. A TLS, X.509, hardware, or incremental cryptography handle wraps native state whose lifetime is controlled through the wrapper's owner. The presence of a Rust method is not proof that a compatible native DLL or shared object is loaded, that its hardware facility is available, or that a remote peer will interoperate.

3: References and Conventions

3.1 Primary project sources

- qsc/src/lib.rs — Public safe crate modules and root availability functions.
- qsc/src/*.rs — API implementations, Rustdoc comments, safety invariants, native status conversion and resource owner types.
- qsc-sys/src/lib.rs — Low-level FFI module graph and exposed native Rust types.
- qsc-sys/src/loader.rs — Windows/Linux/macOS dynamic loader, native symbol table and load diagnostics.
- qsc-sys/src/abi010_frozen_rust_signatures.json — Frozen eager and additive native Rust type/signature inventory.
- qsc-sys/src/abi010_public_symbols.txt — 329-name authoritative ABI 0.10 public C symbol manifest included in the archive.
- scripts/check_abi010_contract.py — Non-executed static contract verification script for exported names and native call signatures.
- qsc/tests/stage1_* through stage15_* — Integration tests, optional native checks and negative cases.
- qsc_examples/src/examples/*.rs — Runnable example demonstrations grouped by stage.
- qsc_tests/src/main.rs — Standalone interactive/native test launcher.

- scripts/vs-build.cmd and QSCSDKRS/QSCSDKRS.vcxproj — Windows Visual Studio NMake/Cargo integration and environment handling.

3.2 External reference links

Rust language and API documentation — https://doc.rust-lang.org/std/

Rust Cargo package and workspace documentation — https://doc.rust-lang.org/cargo/guide/

Rust unsafe code guidelines — https://rust-lang.github.io/unsafe-code-guidelines/

QSC cryptographic SDK repository — https://github.com/Steppenwolfe65/QSC

NIST FIPS 203 — ML-KEM — https://doi.org/10.6028/NIST.FIPS.203

NIST FIPS 204 — ML-DSA — https://doi.org/10.6028/NIST.FIPS.204

NIST FIPS 205 — SLH-DSA — https://doi.org/10.6028/NIST.FIPS.205

RFC 8446 — TLS 1.3 — https://www.rfc-editor.org/rfc/rfc8446

RFC 5280 — X.509 PKI — https://www.rfc-editor.org/rfc/rfc5280

3.3 Contract conventions

Functions and types are rendered in their Rust spelling. slice and Vec lengths are bytes unless otherwise documented. An output-capacity argument is the writable storage size, not proof of produced output length. QscResult<T> means Result<T, QscError>. Except for explicit informational/availability methods, native operations return a Result that must be checked before using output. A C ABI name and a Rust method name should not be conflated.

4: Architecture and Crate Organization

4.1 Call layers

```
Rust application / embedded integrator
   ↓ safe qsc crate: slices, owned Vec, SecretBytes, opaque types
qsc::... public modules (version, hash, symmetric, TLS, X.509...)
   ↓ checked unsafe boundary and native status conversion
qsc-sys crate: lazy dynamic loader + raw QSCABI call signatures
   ↓ platform dynamic linker and 329-symbol ABI 0.10 contract
external QSCABI shared library
   ↓
external QSC native cryptographic implementation
```

4.2 Crate ownership

Component	Role	Integration rule
qsc	Customer-facing safe Rust façade.	Prefer these APIs for all normal application cryptographic operations.
qsc-sys	Low-level, unsafe dynamic loader and raw C ABI arguments.	Use only when implementing or auditing the safe wrapper.
qsc_examples	Standalone examples with stage-specific commands.	Do not treat example fixtures or simple roundtrips as product qualification.
qsc_tests	Standalone interactive/CI-oriented test launcher.	Native availability and strict ABI mode influence test outcomes.
qsc/tests	Rust cargo integration tests.	Required-native tests and feature availability may vary by machine.
QSCSDKRS	Visual Studio 2022 Makefile/NMake frontend.	Requires a valid workspace manifest, cargo/rustc, and native DLL.
scripts	ABI signature checks and Windows Cargo orchestration.	Review before release; do not modify protected external QSC dependencies.

Table 4.1 Crate and tooling responsibilities

4.3 Dynamic native availability

Unlike a static Rust linking strategy, qsc-sys resolves the QSCABI shared library at runtime. Native loader errors must be distinguished from a QSCABI operation returning an error status. The public crate offers qsc::native_available() and qsc::native_load_error(), and qsc::abi_contract::require_full_abi() checks version and manifest completeness.

5: Build and Native Runtime Loading

5.1 Rust workspace requirements

Each member manifest inherits edition, authors, license, repository and lint settings using workspace = true. Therefore the top-level workspace Cargo.toml is required for Cargo metadata, builds and tests. Although referenced by scripts and Visual Studio project metadata, that root manifest is absent from this ZIP. The following commands describe the intended restored workspace, not independently successful commands on the supplied archive.

```
# From a complete QSCRS workspace root with Cargo.toml:
cargo metadata --no-deps --format-version 1
cargo build --workspace
cargo test --workspace
cargo run -p qsc_examples -- -list
cargo run -p qsc_tests -- --test all --no-pause
```

5.2 Native QSCABI deployment

The wrapper does not bundle QSCABI or the QSC static/runtime dependency. Deploy a QSCABI 0.10 compatible native binary compiled with the intended QSC profile and CPU intrinsics support. The loader supports an explicit QSCABI_PATH environment setting as well as its platform search paths; a successful load must still be followed by an ABI compatibility and symbol audit.

The Windows build script writes logs below %LOCALAPPDATA%\QRCS\QSCSDKRS\build-logs, not into protected system folders. It verifies toolchain availability and Cargo metadata before executing the workspace build. Treat build-log directory creation, SDK path discovery, and DLL search as deployment requirements rather than promises of runtime success.

5.3 ABI contract checker

```
python scripts/check_abi010_contract.py
# This is a source-contract inspection, not a native crypto test.
# Runtime acceptance also requires the installed DLL/shared object
# and qsc::abi_contract::require_full_abi().
```

The contract checker reads the frozen symbol/signature files. Its source explicitly expects 252 eager function-pointer fields and 77 additional FFI declarations (17 legacy and 60 additive) for 329 unique native functions. A changed header or matching name is insufficient when the corresponding calling convention or argument type changes.

5.4 Target architecture and cross-platform deployment

Build and run on the intended Windows/MSVC, Linux/GCC-compatible native libraries, and macOS/Clang targets. On a processor lacking AVX-512, a binary containing unguarded AVX-512 instructions may fail at runtime even if Rust code compiles. CPU selection is a QSC native packaging decision; QSCRS does not add an automatic QSC intrinsics dispatcher. A documented per-platform ABI and instruction-set profile is needed for every shipped shared-library variant.

6: Errors, Status and Ownership

6.1 QscError and QscResult

Rust error variant	Meaning / recommended handling
NativeLibraryUnavailable(String)	Native library, symbol, or load path is unavailable; distinguish from operation failure.

Rust error variant	Meaning / recommended handling
NullNativeString(&str)	Expected native C string pointer was NULL; refuse to format it.
TextEncodingFailure(&str)	Native diagnostic text could not be decoded as expected UTF-8.
InvalidArgument(&str)	Caller parameters or option combination rejected before/surrounding FFI.
BufferTooSmall { parameter, required, actual }	Preflight capacity rejected; enlarge caller storage as appropriate.
Status(QscStatus)	Native C ABI status is non-success; preserve the original status category.

Table 6.1 Public QscError model

6.2 SecretBytes lifecycle

SecretBytes wraps a Vec<u8> and intentionally does not implement Clone. It redacts its Debug output and clears its owned bytes in Drop using volatile writes plus a compiler fence. The value owns only its current allocation; other copies, into_bytes transfers and buffers written by callers are not automatically erased. SecretBytes::into_bytes() transfers wiping responsibility to the receiver.

```
use qsc::secret::SecretBytes;
let secret = SecretBytes::new(vec![0x7Au8; 32]);
assert_eq!(secret.len(), 32);
// Avoid printing, cloning, or copying secret.as_slice().
// Drop clears the owned allocation; use of into_bytes() transfers custody.
```

6.3 FFI, threading and drop

The safe wrapper uses owned Rust values and opaque NonNull native state handles. Streaming, TLS, and hardware handle types implement single-thread ownership using PhantomData<Rc<()>> where applicable; they should not be treated as Send or Sync. Drop invokes best-effort native cleanup but cannot turn an earlier failed authentication or operation into a success. Explicit close/shutdown should be used when an application must inspect cleanup status.

6.4 Output capacities and error order

An allocating API returns an owned Vec or SecretBytes only after its checks succeed. _into variants write into supplied mutable slices, usually with preflight lengths and a result containing the exact output count. Do not read data after a failed native call. In particular, an AEAD decrypt operation must not reveal plaintext before tag validation; wrapper-side output isolation and caller-side verification remain distinct requirements.

7: Version and Frozen ABI Contract

7.1 Version and capability queries

version.rs exposes native ABI major, minor and patch, native QSC version text, compiler/platform features, compiled feature bits, native build backend and profile. Always check compatibility before calling an additive ABI 0.10 function; a 329-name manifest check is stronger than a simple major/minor predicate.

```
use qsc::{native_available, native_load_error};
use qsc::abi_contract::require_full_abi;
if !native_available() {
    eprintln!("Native QSCABI load error: {:?}", native_load_error());
} else {
    require_full_abi()?;
    // Use the API only after the required native contract is confirmed.
}
```

7.2 Frozen signatures and migration safety

qsc-sys/src/abi010_frozen_rust_signatures.json records the expected 252 eager native pointer signatures and 77 additional functions, paired with the exact 329-symbol name list. The script/scripts and stage15 tests exist to detect a previous class of regression in which public documentation changes accidentally lost native function definitions. Never regenerate these files by guessing signatures; compare against the authoritative QSCABI headers and compiled exports.

7.3 Availability versus qualification

qsc::native_available only reflects ability to load the native runtime. require_full_abi checks the naming/version contract; it does not establish hardware key custody, cryptographic test-vector compliance, certificate trust or TLS interoperability. Library selection and test qualification remain separate gates.

8: Hash, Encoding and XOF APIs

8.1 Hex, Base64 and PEM

encoding::hex_encode / hex_decode, base64_encode / base64_decode and pem_encode / pem_decode convert Rust data to/from native ABI representations. These functions return Rust Strings or Vecs; the caller must apply a protocol-defined encoding,

allocation limit, and trust policy. A syntactically valid PEM container is not a trusted certificate.

8.2 SHA-2 and SHA-3

hash.rs uses macros to instantiate fixed-length SHA2-256/384/512 and SHA3-128/256/512 helpers, each with a digest size query, allocating compute function, and buffer-writing _into function. The digest functions operate on &[u8], do not consume the message, and signal insufficient destination lengths through QscError::BufferTooSmall. Hashes alone do not authenticate messages.

```
use qsc::hash;
let digest = hash::sha2_256(b"message")?;
assert_eq!(digest.len(), hash::sha2_256_digest_size()?);
```

8.3 SHAKE/cSHAKE

xof.rs generates user-specified-length SHAKE128/256/512 output. It requires a nonzero output length or nonempty destination. The streaming module adds cSHAKE with function name and customization domain-separation bytes. For long messages or multiple output blocks, use an owned Xof state with absorb, finalize, then squeeze. Do not treat an XOF as a keyed message authenticator.

9: MAC and Key Derivation

9.1 MAC families

mac.rs supplies HMAC-SHA2, KMAC and KMAC-XOF, Poly1305, QMAC and PBMAC1, including corresponding output-size queries and caller-buffer alternatives. KMAC custom strings are cryptographic domain separation and must match peer byte-for-byte. Poly1305 key material is one-time; never reuse a one-time MAC key with more than one record.

9.2 Key derivation

kdf.rs exposes HKDF SHA2 extract/expand and PBMAC1 key derivation, including _into variants. Input key material, salt, info string and output length represent distinct values. An output-buffer check is a safety condition, not proof the derivation inputs are sufficiently unpredictable. Password-based derivation requires an independently reviewed iteration and salt policy.

9.3 Authenticated comparison

Successful HMAC or KMAC calculation produces a tag; verification is a separate decision. Use native memory::are_equal for equal-length secret comparison when appropriate and reject unequal lengths before comparing. Where the wrapper exposes a verified form, consume its status/result rather than reimplementing a potentially variable-time comparison.

10: Symmetric Encryption and AEAD

10.1 Supported algorithm families

Family	Rust facade	Security note
AES GCM	aes128_gcm_*, aes256_gcm_*	AEAD with separate nonce and AAD; use checked authenticated decrypt.
ChaCha20-Poly1305	chacha20_poly1305_*	AEAD; generated/validated tag and unique per-key nonce.
AES CTR	aes256_ctr_transform*	Confidentiality only; native endianness is explicitly selectable.
ChaCha stream cipher	chacha20_*	Unauthenticated; transport must supply message integrity.
CSX and RCS	csx_* and rcs_*	Native QRCS authenticated ciphers; profile-dependent key and tag sizing.

Table 10.1 Symmetric family mapping

10.2 Owned and caller-buffer forms

The allocating forms return Vec<u8>. Corresponding _into operations accept a mutable destination and return the number of bytes written when an output length may vary. Checked sizing and out-of-place buffers prevent accidental overwrite. Ciphertext output often includes a tag; do not truncate to the plaintext length.

10.3 Nonce and authentication policy

Never reuse a nonce under the same AEAD key. Record nonce generation and rekeying rules in the application protocol. If authentication fails, drop the output and do not continue using a mutated cryptographic state. CTR and ChaCha stream transformations alone provide no integrity or replay protection.

11: Post-Quantum Key Encapsulation

11.1 KEM key pair and encapsulation objects

kem.rs represents KEM public/private key pairs as KemKeyPair and encapsulation output as KemEncapsulation. Private keys and shared secrets reside in SecretBytes. The facade includes ML-KEM, HQC and Classic McEliece with size helpers and standard keygen/encapsulate/decapsulate sequences selected by the linked QSC build.

```
use qsc::kem::mlkem;
let pair = mlkem::generate_keypair()?;
let sealed = mlkem::encapsulate(&pair.public_key)?;
let opened = mlkem::decapsulate(&sealed.ciphertext, pair.private_key.as_slice())?;
// Compare only after each result has succeeded; authenticate peers separately.
```

11.2 Implicit rejection and authentication

A KEM shared secret does not establish the identity of the peer. Under implicit rejection, a malformed encapsulation may still yield secret bytes as a side-channel mitigation, so protocol-level peer authentication and transcript binding are required. Buffer length correctness, public-key provenance and negotiated algorithm identifiers must be enforced by the application.

12: Digital Signatures and Classical Curves

12.1 ML-DSA, Falcon, SLH-DSA and classical signatures

signature.rs groups signed-message operations under named submodules. A signature verify API returning Vec<u8> may recover the authenticated message in the attached-message format. It is not always a Boolean verifier. Applications must distinguish detached and attached encodings and use the precise public-key and signed-message size helpers. Example commands currently comment out the Falcon-specific menu alias even though wrapper source declares Falcon operations.

12.2 ECDH/EDDH key agreement

asymmetric.rs exposes ECDH and EDDH key generation, seeded generation and shared-secret agreement through the native library. The returned shared secret is SecretBytes; derive application keys using protocol-defined KDF inputs. Successful agreement proves mathematical consistency, not remote identity.

12.3 ECDSA and EdDSA representation

ECDSA and EdDSA sign/verify functions have parameter sizes determined by the selected native profile. Do not assume raw QSC signatures are interchangeable with DER-encoded OpenSSL signatures. Preserve signature and message encoding across peers and validate application authorization after successful cryptographic verification.

13: Explicit Algorithm Parameter APIs

13.1 Runtime selectors

explicit_kem.rs, explicit_curve.rs, explicit_signature.rs and explicit_slhdsa.rs accept named enum selectors and query their sizes before key generation, signing or encapsulation. This makes runtime parameter selection visible in Rust rather than hidden behind a build-time QSC native default. Selection remains subject to actual native capabilities and the deployed QSC build.

13.2 SLH-DSA context and prehash selection

The explicit SLH-DSA API accepts configured algorithm, prehash mode, randomization policy and context string, with sizes returned by the native ABI. The source contracts constrain context bytes and supported prehash combinations. Unsupported parameter combinations must fail closed; the wrapper must not silently substitute a different parameter set.

13.3 Availability checks

Each explicit selector module exposes available() and sizes(...); available() indicates that a native symbol path exists, not that every algorithm variant can execute. Query sizes and status for the chosen enum and keep the selected identifier bound to the protocol transcript for interoperability and downgrade resistance.

14: Streaming and Stateful Cryptography

14.1 Single-thread-owned state objects

Rust handle	Typical sequence	Failure discipline
Digest	new → update* → finalize → Drop	Finalize once; no further updates; state cleanup on Drop.
Xof	new → absorb* → finalize → squeeze* → Drop	No absorb after finalize; output capped per source.

Rust handle	Typical sequence	Failure discipline
StreamMac	new → update* → finalize → Drop	Use correct key, algorithm, output size; no tag reuse assumptions.
CtrStream	new → transform* → Drop	Confidentiality only; nonce/counter uniqueness and external authentication.

Table 14.1 Streaming state lifecycles

14.2 Lifetime and output bounds

streaming.rs bounds allocating output requests with MAX_OUTPUT_BYTES = 64 MiB. It owns native NonNull handles and uses a non-Send/non-Sync marker for the stateful classes. A Rust move transfers object ownership, not permission to duplicate native handles. Native destroy is called by Drop, but applications should still check the finalization result explicitly.

14.3 Streaming MAC, XOF and CTR validation

The integration suite includes separate stage10 streaming hash/XOF and stage11 streaming MAC/CTR tests. For production, test repeated finalize, zero-length inputs and invalid state, buffer capacities, CTR counter exhaustion, and partial records; do not assume all failures imply a recoverable state. Streaming AEAD is not supplied as a general authenticated streaming Rust facade in this source.

15: X.509 Certificates and Issuance

15.1 Certificate, chain and store wrappers

x509.rs defines owned Certificate, Chain and TrustStore wrappers around opaque QSCABI pointers, with explicit load/export/subject/issuer/hostname and verification operations. Rust byte/string input remains borrowed during the synchronous native call, and library-owned objects are freed through Drop. Certificate parsing, hostname matching and path validation are distinct trust decisions.

15.2 CSR and CRL builder APIs

issuance.rs adds Csr and CrlBuilder wrappers with create, configure, sign, export, verify and revocation operations. The QSCABI source offers constrained P-256 issuance functions rather than a general-purpose certificate authority engine. The implementation needs correct DER signature representation, key custody and trusted issuer permissions before issuing any identity.

15.3 Verification limits

Require an independently trusted root, correct intermediates, validity checks, purpose and expected DNS identity. A CSR signature proves integrity but not authority to issue the requested identity. CRL verification must check issuer and freshness as required by the native API; errors and unknown status must not be interpreted as approval.

16: TLS 1.3 and Network Lifecycle

16.1 Policy and connection types

tls.rs exposes TlsPolicy, SignatureScheme, AddressFamily, SocketOptions, TicketPolicy, TlsContext, TlsConnection, TlsListener and TlsServer with additional peer metadata. The native C interface owns network implementation details; public Rust transport handles do not expose QSC socket structures. Configuration is not an authenticated handshake.

16.2 Configuration and call hierarchy

```
let mut context = TlsContext::create()?;
context.set_policy(TlsPolicy::Strict)?;
// Configure ALPN, SNI, trust anchors and identity before connect/bind.
// Create connection or listener/server and negotiate a verified TLS 1.3 session.
// Check authentication and selected peer identity before any app data.
// Use send/receive or framed I/O, shutdown, then release native handles.
```

16.3 File-based credentials and stubs

The facade exposes PEM-named loader methods as well as file-based trust-anchor, identity and CRL operations. The underlying QSCABI 0.10 source retains earlier in-memory PEM compatibility exports that can return Unsupported; implementations must use the functional file-based methods for production. A method’s source-level availability alone is not proof of loaded credentials or trusted peers.

16.4 Blocking and peer-policy requirements

TLS send and receive return explicit status and byte counts. The protocol application must handle partial reads, framing, shutdown and errors. ALPN and SNI are negotiation and routing controls; the application must independently verify the accepted certificate chain and host/service name, and enforce client-authentication policy where enabled.

17: Hardware Provider and Key Custody

17.1 Callback registration and policies

hardware.rs exposes HwPolicy, HwCapabilities, HwKeyInfo and HwBackendOps from qsc-sys. Its policy() helper defaults to require_hardware=1, allow_software_fallback=0, allow_export=0 and allow_derived_secret_export=0. The HardwareProvider::open function is unsafe because callbacks must be valid, non-unwinding C functions and the backend context must remain valid through provider destruction.

17.2 Key lifetime and threading

HardwareProvider and HardwareKey contain native handles with Rust ownership and single-thread markers. A token may become stale after a device disconnect or provider invalidation. Closing a provider with live keys, callback reentrancy, unhandled panics crossing FFI and invalidating backend memory are integration defects; they require device-specific testing.

17.3 Device capability is not device attestation

hardware::is_enabled tests compiled bridge availability, not physical hardware presence, key non-exportability or attestation validity. Hardware policy enforcement, HSM/TPM firmware trust, key custody and certification are outside source-only documentation. Fail closed if hardware storage is mandatory and a compatible provider or device cannot be opened.

18: Rust Examples and Integration Patterns

18.1 Example crate structure

Example module	Main subject
stage2.rs	Encoding, entropy, hash, XOF, MAC and KDF calls.
stage3.rs	AES, ChaCha/Poly1305, CSX, RCS and symmetric interfaces.
stage4.rs	KEMs, PQ signatures and classical asymmetric operations.
stage5.rs	Certificate loading, subjects, identities, hostname and export.
stage6.rs	TLS context, policy, socket, ALPN and lifecycle.
stage7.rs	Explicit parameter, key, curve, signature and SLH-DSA selection.
stage9.rs	ABI capabilities, streaming, file-based TLS, CSR/CRL and hardware.
commands.rs and main.rs	CLI dispatcher, command help and interactive REPL examples.

Table 18.1 Uploaded Rust examples

18.2 Example commands

```
# After restoring a complete Cargo workspace:
cargo run -p qsc_examples -- -list
cargo run -p qsc_examples -- -test hash
cargo run -p qsc_examples -- -test symmetric
cargo run -p qsc_examples -- -test pqc
cargo run -p qsc_examples -- -test tls
cargo run -p qsc_examples -- -test stream-hash
cargo run -p qsc_examples -- -test csr
```

The interactive runner accepts individual groups; an empty argument list enters the help/REPL. Demonstration operations may use mock fixtures or stop before connecting to a live peer. They must not be interpreted as hardware qualification, remote TLS interoperability or successful release testing.

18.3 Recommended integration sequence

- Restore and verify a valid workspace manifest and loadable native QSCABI binary.
- Check ABI compatibility and 329-name contract before optional algorithms.
- Use qsc-level safe methods rather than direct qsc-sys pointers.
- Select algorithms and obtain current native size results; allocate separately owned buffers.
- Propagate QscResult errors; do not unwrap cryptographic errors on network paths.
- Apply application authentication, certificate trust, nonce policy and private-key storage.
- Run source, managed and native integration suites on each target platform.

19: Tests and Qualification Evidence

19.1 Staged Cargo integration suites

The archive contains Rust integration tests stage1 through stage15 covering core, encoding, cipher, asymmetric, X.509, TLS, explicit KEM/SLH-DSA, capabilities, streaming, file credential loading, CSR issuance, hardware providers and export-contract verification. Stage15 verifies the frozen 329-symbol set. Individual integration tests cannot substitute for application-level multi-host and physical-device testing.

19.2 Interactive test runner and CI switches

qsc_tests exposes a CLI with --test <group>, --no-pause, --strict-abi and --require-native options. It is distinct from Cargo’s test harness. The executable should report unique PASS/FAIL diagnostics for each named test and a useful final status. Its native

test outcome is sensitive to missing/old QSCABI binary and to the environment where it executes.

19.3 Evidence limit

The uploaded ZIP includes Cargo target/debug incremental metadata and .fingerprint records. These are historical compilation artifacts, not a clean successful build or qualification record for the present source. No workspace-root Cargo.toml was included, and neither Cargo nor the native ABI was exercised for this specification. Treat all test and build outcomes as not executed unless separately supported by fresh logs.

20: Deployment, Limitations and Security Review

20.1 Operational limits

- QSCABI is an independently provided runtime dependency; loading a shared binary is a deployment and supply-chain decision.
- The source archive is missing the workspace-root Cargo.toml; restore its matching authorized root manifest before any full build.
- Version 0.2.0 is the Rust package version and must not be confused with native QSCABI 0.10 or QSC library revision.
- Use compatible scalar/AES-NI/AVX processor variants; Rust does not automatically select QSC optimized native binaries.
- The 329-symbol contract is necessary for wrapper coverage but not sufficient for algorithm correctness, certification or peer identity.
- Hardware callbacks must be C-compatible, non-panicking, lifecycle-safe and thread-synchronization aware.
- Sensitive key bytes must be erased from all owned and copied buffers, not just SecretBytes.
- Do not treat attached signatures or KEM secrets as authorization without separate peer and transcript validation.

20.2 Recommended release gates

Gate	Implementation evidence
Workspace and toolchain	Metadata parse, Cargo build, platform toolchain and native DLL/shared object identity.
Frozen native contract	Exact 329 exports and 252+77 signature agreement; no obsolete QSCABI symbols.
Safety	No double frees, use-after-close, callback unwinds, secret-bearing Debug logs or data races.

Gate	Implementation evidence
Cryptographic behavior	Known-answer, roundtrip, bounds, authentication-failure and algorithm-specific negative tests.
TLS and X.509	Trusted issuer, hostname, client auth, SNI/ALPN, file loaders and live peers.
Hardware	Physical device, disconnection, provider policy, invalidation and attestation results.
Distribution	QRCS-CASL inclusion, platform packaging, signing/hash manifests and installed-app smoke test.

Table 20.1 Release qualification gates

21: Complete Public Rust API Reference

The following pages reproduce 378 public Rust function signatures discovered in qsc/src, including the constructors and methods declared on public types and the public functions generated by visible macro invocations. Rustdoc summary sentences are included where present. A macro-generated entry is explicitly labeled, with the QSCABI call supplied by the invocation. Private helper functions, trait implementations without public method declarations, and non-Rust native symbols do not enlarge this list.

21.1 abi_contract.rs (3 public functions)

abi_contract::expected_exports

pub fn expected_exports() -> &'static str

Purpose: Return the exact expected public symbol table bundled from QSCABI 0.10.

abi_contract::missing_exports

pub fn missing_exports() -> QscResult<Vec<String>>

Purpose: List the names absent from the currently loaded native QSCABI shared library.

abi_contract::require_full_abi

pub fn require_full_abi() -> QscResult<()>

Purpose: Require compatible major/minor and all 329 public C export names.

21.2 asymmetric.rs (16 public functions)

asymmetric::ecdh::generate_keypair

pub fn generate_keypair() -> QscResult<AsymmetricKeyPair>

Purpose: Generates an ECDH key pair.

asymmetric::ecdh::generate_seeded_keypair

pub fn generate_seeded_keypair(seed: &[u8]) -> QscResult<AsymmetricKeyPair>

Purpose: Generates a deterministic ECDH key pair from a seed.

asymmetric::ecdh::public_from_private

pub fn public_from_private(private_key: &[u8]) -> QscResult<Vec<u8>>

Purpose: Derives the ECDH public key from a private key.

asymmetric::ecdh::key_exchange

pub fn key_exchange(private_key: &[u8], public_key: &[u8]) -> QscResult<SecretBytes>

Purpose: Performs ECDH key exchange.

asymmetric::eddh::generate_keypair

pub fn generate_keypair() -> QscResult<AsymmetricKeyPair>

Purpose: Generates an EDDH key pair.

asymmetric::eddh::generate_seeded_keypair

pub fn generate_seeded_keypair(seed: &[u8]) -> QscResult<AsymmetricKeyPair>

Purpose: Generates a deterministic EDDH key pair from a seed.

asymmetric::eddh::public_from_private

pub fn public_from_private(private_key: &[u8]) -> QscResult<Vec<u8>>

Purpose: Derives the EDDH public key from a private key.

asymmetric::eddh::key_exchange

pub fn key_exchange(private_key: &[u8], public_key: &[u8]) -> QscResult<SecretBytes>

Purpose: Performs EDDH key exchange.

asymmetric::ecdh::public_key_size

pub fn public_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::asymmetric::qscabi_ecdh_public_key_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::asymmetric::qscabi_ecdh_public_key_size. Return status and capacity are checked by the macro implementation.

asymmetric::ecdh::private_key_size

pub fn private_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::asymmetric::qscabi_ecdh_private_key_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::asymmetric::qscabi_ecdh_private_key_size. Return status and capacity are checked by the macro implementation.

asymmetric::ecdh::shared_secret_size

pub fn shared_secret_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::asymmetric::qscabi_ecdh_shared_secret_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::asymmetric::qscabi_ecdh_shared_secret_size. Return status and capacity are checked by the macro implementation.

asymmetric::ecdh::seed_size

pub fn seed_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::asymmetric::qscabi_ecdh_seed_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::asymmetric::qscabi_ecdh_seed_size. Return status and capacity are checked by the macro implementation.

asymmetric::eddh::public_key_size

pub fn public_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::asymmetric::qscabi_eddh_public_key_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::asymmetric::qscabi_eddh_public_key_size. Return status and capacity are checked by the macro implementation.

asymmetric::eddh::private_key_size

pub fn private_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::asymmetric::qscabi_eddh_private_key_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::asymmetric::qscabi_eddh_private_key_size. Return status and capacity are checked by the macro implementation.

asymmetric::eddh::shared_secret_size

pub fn shared_secret_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::asymmetric::qscabi_eddh_shared_secret_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::asymmetric::qscabi_eddh_shared_secret_size. Return status and capacity are checked by the macro implementation.

asymmetric::eddh::seed_size

pub fn seed_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::asymmetric::qscabi_eddh_seed_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::asymmetric::qscabi_eddh_seed_size. Return status and capacity are checked by the macro implementation.

21.3 encoding.rs (8 public functions)

encoding::hex_encoded_size

pub fn hex_encoded_size(input_length: usize) -> QscResult<usize>

Purpose: Returns the hexadecimal encoded character count, excluding the terminator.

encoding::hex_encode

pub fn hex_encode(data: &[u8]) -> QscResult<String>

Purpose: Encodes binary data as uppercase hexadecimal text.

encoding::hex_decode

pub fn hex_decode(hex: &str) -> QscResult<Vec<u8>>

Purpose: Decodes hexadecimal text into binary data.

encoding::base64_encoded_size

pub fn base64_encoded_size(input_length: usize) -> QscResult<usize>

Purpose: Returns the Base64 encoded character count, excluding the terminator.

encoding::base64_encode

pub fn base64_encode(data: &[u8]) -> QscResult<String>

Purpose: Encodes binary data as RFC 4648 Base64 text.

encoding::base64_decode

pub fn base64_decode(base64: &str) -> QscResult<Vec<u8>>

Purpose: Decodes RFC 4648 Base64 text into binary data.

encoding::pem_encode

pub fn pem_encode(label: &str, data: &[u8]) -> QscResult<String>

Purpose: Encodes binary data as PEM armor using the supplied label.

encoding::pem_decode

pub fn pem_decode(pem: &str) -> QscResult<Vec<u8>>

Purpose: Decodes PEM armor into binary data.

21.4 entropy.rs (14 public functions)

entropy::acp_seed_max

pub fn acp_seed_max() -> QscResult<usize>

Purpose: Returns the ACP provider maximum seed length.

entropy::csp_seed_max

pub fn csp_seed_max() -> QscResult<usize>

Purpose: Returns the CSP provider maximum seed length.

entropy::rdp_seed_max

pub fn rdp_seed_max() -> QscResult<usize>

Purpose: Returns the RDP provider maximum seed length.

entropy::cjp_seed_max

pub fn cjp_seed_max() -> QscResult<usize>

Purpose: Returns the CJP provider maximum seed length.

entropy::is_rdp_available

pub fn is_rdp_available() -> QscResult<bool>

Purpose: Returns whether the RDP provider is available on this platform.

entropy::is_cjp_available

pub fn is_cjp_available() -> QscResult<bool>

Purpose: Returns whether the CJP provider is available on this platform.

entropy::generate_acp

pub fn generate_acp(length: usize) -> QscResult<Vec<u8>>

Purpose: Generates ACP entropy into a new vector.

entropy::generate_csp

pub fn generate_csp(length: usize) -> QscResult<Vec<u8>>

Purpose: Generates CSP entropy into a new vector.

entropy::generate_rdp

pub fn generate_rdp(length: usize) -> QscResult<Vec<u8>>

Purpose: Generates RDP entropy into a new vector.

entropy::generate_cjp

pub fn generate_cjp(length: usize) -> QscResult<Vec<u8>>

Purpose: Generates CJP entropy into a new vector.

entropy::generate_acp_into

pub fn generate_acp_into(output: &mut [u8]) -> QscResult<()>

Purpose: Fills a caller-owned buffer with ACP entropy.

entropy::generate_csp_into

pub fn generate_csp_into(output: &mut [u8]) -> QscResult<()>

Purpose: Fills a caller-owned buffer with CSP entropy.

entropy::generate_rdp_into

pub fn generate_rdp_into(output: &mut [u8]) -> QscResult<()>

Purpose: Fills a caller-owned buffer with RDP entropy.

entropy::generate_cjp_into

pub fn generate_cjp_into(output: &mut [u8]) -> QscResult<()>

Purpose: Fills a caller-owned buffer with CJP entropy.

21.5 explicit_curve.rs (6 public functions)

explicit_curve::available

pub fn available() -> bool

Purpose: Test for the presence of an explicit named-curve ABI symbol.

explicit_curve::sizes

pub fn sizes(algorithm: CurveAlgorithm) -> QscResult<CurveSizes>

Purpose: Obtain lengths required for one curve, rejecting invalid metadata.

explicit_curve::generate

pub fn generate(algorithm: CurveAlgorithm) -> QscResult<AsymmetricKeyPair>

Purpose: Generate a new native curve keypair, with private scalar zeroed on drop.

explicit_curve::generate_seeded

pub fn generate_seeded(algorithm: CurveAlgorithm, seed: &[u8]) -> QscResult<AsymmetricKeyPair>

Purpose: Deterministically derive a keypair from an exactly sized seed.

explicit_curve::public_from_private

pub fn public_from_private(algorithm: CurveAlgorithm, private_key: &[u8]) -> QscResult<Vec<u8>>

Purpose: Derive the public point from a matching private key of exact length.

explicit_curve::agree

pub fn agree(algorithm: CurveAlgorithm, private_key: &[u8], peer_public: &[u8]) -> QscResult<SecretBytes>

Purpose: Agree on a candidate shared secret using a peer public point and local key.

21.6 explicit_kem.rs (5 public functions)

explicit_kem::available

pub fn available() -> bool

Purpose: Return true if the loaded native library exports explicit KEM metadata.

explicit_kem::sizes

pub fn sizes(algorithm: KemAlgorithm) -> QscResult<KemSizes>

Purpose: Return the exact native byte lengths for a named parameter set.

explicit_kem::generate

pub fn generate(algorithm: KemAlgorithm) -> QscResult<KemKeyPair>

Purpose: Generate an independently selected KEM keypair.

explicit_kem::encapsulate

pub fn encapsulate(algorithm: KemAlgorithm, public_key: &[u8]) -> QscResult<KemEncapsulation>

Purpose: Encapsulate to a validated-length public key; return ciphertext and secret.

explicit_kem::decapsulate

pub fn decapsulate(algorithm: KemAlgorithm, ciphertext: &[u8], private_key: &[u8]) -> QscResult<SecretBytes>

Purpose: Decapsulate an authenticated peer ciphertext into a secret container.

21.7 explicit_signature.rs (5 public functions)

explicit_signature::available

pub fn available() -> bool

Purpose: Whether the loaded QSCABI exports the newer signature-size function.

explicit_signature::sizes

pub fn sizes(algorithm: SignatureAlgorithm) -> QscResult<SignatureSizes>

Purpose: Query exact signing key lengths and maximum attached-signature overhead.

explicit_signature::generate

pub fn generate(algorithm: SignatureAlgorithm) -> QscResult<SignatureKeyPair>

Purpose: Generate a fresh keypair for the selected attached-signature scheme.

explicit_signature::sign

pub fn sign(algorithm: SignatureAlgorithm, message: &[u8], private_key: &[u8]) -> QscResult<Vec<u8>>

Purpose: Return an attached signed-message encoding of the supplied message bytes.

explicit_signature::verify

pub fn verify(algorithm: SignatureAlgorithm, signed: &[u8], public_key: &[u8]) -> QscResult<Vec<u8>>

Purpose: Verify an attached signature and recover only authenticated message bytes.

21.8 explicit_slhdsa.rs (5 public functions)

explicit_slhdsa::available

pub fn available() -> bool

Purpose: Return whether the native library exports the explicit SLH-DSA size query.

explicit_slhdsa::sizes

pub fn sizes(algorithm: SlhDsaAlgorithm) -> QscResult<SlhDsaSizes>

Purpose: Retrieve exact byte lengths, rejecting impossible or unbounded metadata.

explicit_slhdsa::generate

pub fn generate(algorithm: SlhDsaAlgorithm) -> QscResult<SlhDsaKeyPair>

Purpose: Generate the selected SLH-DSA keypair with native CSP randomness.

explicit_slhdsa::sign

pub fn sign(algorithm: SlhDsaAlgorithm, prehash: SlhDsaPrehash, randomized: bool, message: &[u8], context: &[u8], private_key: &[u8]) -> QscResult<Vec<u8>>

Purpose: Sign the exact message and up-to-255-byte context, returning a detached tag.

explicit_slhdsa::verify

pub fn verify(algorithm: SlhDsaAlgorithm, prehash: SlhDsaPrehash, signature: &[u8], message: &[u8], context: &[u8], public_key: &[u8]) -> QscResult<()>

Purpose: Verify a detached signature with the matching message/context/prehash.

21.9 hardware.rs (11 public functions)

hardware::is_enabled

pub fn is_enabled() -> QscResult<bool>

Purpose: Query whether native hardware-provider code was compiled in, not connected.

hardware::policy

pub fn policy(algorithm: u32, usage: u32) -> HwPolicy

Purpose: Construct a versioned policy with deny-by-default custody/export settings.

hardware::HardwareProvider::open

pub unsafe fn open(callbacks: &HwBackendOps, context: *mut c_void) -> QscResult<Self>

Purpose: Register a native provider with its externally managed callback context.

Safety: Explicit unsafe Rust boundary; caller must satisfy native pointer/callback validity and lifetime requirements documented in this module.

hardware::HardwareProvider::capabilities

pub fn capabilities(&mut self) -> QscResult<HwCapabilities>

Purpose: Obtain the backend's current capability mask and threading constraints.

hardware::HardwareProvider::invalidate

pub fn invalidate(&mut self) -> QscResult<()>

Purpose: Invalidate any live device key token after a disconnect or rekey event.

hardware::HardwareProvider::random

pub fn random(&mut self, length: usize) -> QscResult<Vec<u8>>

Purpose: Request random bytes from the selected backend; not a FIPS claim.

hardware::HardwareKey<'_>::info

pub fn info(&mut self) -> QscResult<HwKeyInfo>

Purpose: Return public key custody and nonexportability details from the provider.

hardware::HardwareKey<'_>::public_key

pub fn public_key(&mut self) -> QscResult<Vec<u8>>

Purpose: Export public-only key bytes; never attempts to export private key data.

hardware::HardwareKey<'_>::sign

pub fn sign(&mut self, message: &[u8]) -> QscResult<Vec<u8>>

Purpose: Sign borrowed message bytes without exporting the native signing key.

hardware::HardwareKey<'_>::agree

pub fn agree(&mut self, peer: &[u8]) -> QscResult<SecretBytes>

Purpose: Derive a raw secret only when the configured backend policy permits export.

hardware::HardwareKey<'_>::attest

pub fn attest(&mut self, challenge: &[u8]) -> QscResult<Vec<u8>>

Purpose: Obtain an opaque attestation evidence blob; independent validation required.

21.10 hash.rs (18 public functions)

hash::sha2_256_digest_size

pub fn sha2_256_digest_size() -> QscResult<usize>

Purpose: Macro-generated public digest_size_fn wrapper; refer to the native binding qsc_sys::hash::qscabi_sha2_256_digest_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::hash::qscabi_sha2_256_digest_size. Return status and capacity are checked by the macro implementation.

hash::sha2_384_digest_size

pub fn sha2_384_digest_size() -> QscResult<usize>

Purpose: Macro-generated public digest_size_fn wrapper; refer to the native binding qsc_sys::hash::qscabi_sha2_384_digest_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::hash::qscabi_sha2_384_digest_size. Return status and capacity are checked by the macro implementation.

hash::sha2_512_digest_size

pub fn sha2_512_digest_size() -> QscResult<usize>

Purpose: Macro-generated public digest_size_fn wrapper; refer to the native binding qsc_sys::hash::qscabi_sha2_512_digest_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::hash::qscabi_sha2_512_digest_size. Return status and capacity are checked by the macro implementation.

hash::sha3_128_digest_size

pub fn sha3_128_digest_size() -> QscResult<usize>

Purpose: Macro-generated public digest_size_fn wrapper; refer to the native binding qsc_sys::hash::qscabi_sha3_128_digest_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::hash::qscabi_sha3_128_digest_size. Return status and capacity are checked by the macro implementation.

hash::sha3_256_digest_size

pub fn sha3_256_digest_size() -> QscResult<usize>

Purpose: Macro-generated public digest_size_fn wrapper; refer to the native binding qsc_sys::hash::qscabi_sha3_256_digest_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::hash::qscabi_sha3_256_digest_size. Return status and capacity are checked by the macro implementation.

hash::sha3_512_digest_size

pub fn sha3_512_digest_size() -> QscResult<usize>

Purpose: Macro-generated public digest_size_fn wrapper; refer to the native binding qsc_sys::hash::qscabi_sha3_512_digest_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::hash::qscabi_sha3_512_digest_size. Return status and capacity are checked by the macro implementation.

hash::sha2_256

pub fn sha2_256(message: &[u8]) -> QscResult<Vec<u8>>

Purpose: Macro-generated public digest_fns wrapper; refer to the native binding qsc_sys::hash::qscabi_sha2_256_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::hash::qscabi_sha2_256_compute. Return status and capacity are checked by the macro implementation.

hash::sha2_256_into

pub fn sha2_256_into(message: &[u8], digest: &mut [u8]) -> QscResult<()>

Purpose: Macro-generated public digest_fns wrapper; refer to the native binding qsc_sys::hash::qscabi_sha2_256_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::hash::qscabi_sha2_256_compute. Return status and capacity are checked by the macro implementation.

hash::sha2_384

pub fn sha2_384(message: &[u8]) -> QscResult<Vec<u8>>

Purpose: Macro-generated public digest_fns wrapper; refer to the native binding qsc_sys::hash::qscabi_sha2_384_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::hash::qscabi_sha2_384_compute. Return status and capacity are checked by the macro implementation.

hash::sha2_384_into

pub fn sha2_384_into(message: &[u8], digest: &mut [u8]) -> QscResult<()>

Purpose: Macro-generated public digest_fns wrapper; refer to the native binding qsc_sys::hash::qscabi_sha2_384_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::hash::qscabi_sha2_384_compute. Return status and capacity are checked by the macro implementation.

hash::sha2_512

pub fn sha2_512(message: &[u8]) -> QscResult<Vec<u8>>

Purpose: Macro-generated public digest_fns wrapper; refer to the native binding qsc_sys::hash::qscabi_sha2_512_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::hash::qscabi_sha2_512_compute. Return status and capacity are checked by the macro implementation.

hash::sha2_512_into

pub fn sha2_512_into(message: &[u8], digest: &mut [u8]) -> QscResult<()>

Purpose: Macro-generated public digest_fns wrapper; refer to the native binding qsc_sys::hash::qscabi_sha2_512_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::hash::qscabi_sha2_512_compute. Return status and capacity are checked by the macro implementation.

hash::sha3_128

pub fn sha3_128(message: &[u8]) -> QscResult<Vec<u8>>

Purpose: Macro-generated public digest_fns wrapper; refer to the native binding qsc_sys::hash::qscabi_sha3_128_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::hash::qscabi_sha3_128_compute. Return status and capacity are checked by the macro implementation.

hash::sha3_128_into

pub fn sha3_128_into(message: &[u8], digest: &mut [u8]) -> QscResult<()>

Purpose: Macro-generated public digest_fns wrapper; refer to the native binding qsc_sys::hash::qscabi_sha3_128_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::hash::qscabi_sha3_128_compute. Return status and capacity are checked by the macro implementation.

hash::sha3_256

pub fn sha3_256(message: &[u8]) -> QscResult<Vec<u8>>

Purpose: Macro-generated public digest_fns wrapper; refer to the native binding qsc_sys::hash::qscabi_sha3_256_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::hash::qscabi_sha3_256_compute. Return status and capacity are checked by the macro implementation.

hash::sha3_256_into

pub fn sha3_256_into(message: &[u8], digest: &mut [u8]) -> QscResult<()>

Purpose: Macro-generated public digest_fns wrapper; refer to the native binding qsc_sys::hash::qscabi_sha3_256_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::hash::qscabi_sha3_256_compute. Return status and capacity are checked by the macro implementation.

hash::sha3_512

pub fn sha3_512(message: &[u8]) -> QscResult<Vec<u8>>

Purpose: Macro-generated public digest_fns wrapper; refer to the native binding qsc_sys::hash::qscabi_sha3_512_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::hash::qscabi_sha3_512_compute. Return status and capacity are checked by the macro implementation.

hash::sha3_512_into

pub fn sha3_512_into(message: &[u8], digest: &mut [u8]) -> QscResult<()>

Purpose: Macro-generated public digest_fns wrapper; refer to the native binding qsc_sys::hash::qscabi_sha3_512_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::hash::qscabi_sha3_512_compute. Return status and capacity are checked by the macro implementation.

21.11 issuance.rs (13 public functions)

issuance::valid_time

pub fn valid_time(time: &X509Time) -> bool

Purpose: Validate RFC 5280 UTC calendar components prior to native signing.

issuance::Csr::new

pub fn new() -> QscResult<Self>

Purpose: Allocate an empty native CSR.

issuance::Csr::init_p256

pub fn init_p256(&mut self, common_name: &str, public_key: &[u8]) -> QscResult<()>

Purpose: Initialize with a borrowed 64-byte uncompressed P-256 point coordinate pair.

issuance::Csr::add_dns

pub fn add_dns(&mut self, hostname: &str) -> QscResult<()>

Purpose: Add a nonempty DNS subject-alternative name to the request.

issuance::Csr::load_der

pub fn load_der(&mut self, der: &[u8]) -> QscResult<()>

Purpose: Load a DER CSR into this owned parser state, replacing previous contents.

issuance::Csr::verify

pub fn verify(&self) -> QscResult<()>

Purpose: Cryptographically verify a loaded signed CSR and its public-key proof.

issuance::Csr::sign_der

pub unsafe fn sign_der(&self, signer: X509SignCallback, context: *mut c_void, capacity: usize) -> QscResult<Vec<u8>>

Purpose: Sign an initialized CSR using an externally held private-key callback.

Safety: Explicit unsafe Rust boundary; caller must satisfy native pointer/callback validity and lifetime requirements documented in this module.

issuance::Csr::issue_tls_server_p256

pub unsafe fn issue_tls_server_p256(&self, issuer: &Certificate, serial: &[u8], not_before: &X509Time, not_after: &X509Time, signer: X509SignCallback, context: *mut c_void, capacity: usize) -> QscResult<Vec<u8>>

Purpose: Issue a TLS P-256 leaf certificate from a verified CSR and trusted CA.

Safety: Explicit unsafe Rust boundary; caller must satisfy native pointer/callback validity and lifetime requirements documented in this module.

issuance::CrlBuilder<'a>::new

pub fn new(issuer: &'a Certificate, this_update: &X509Time, next_update: &X509Time) -> QscResult<Self>

Purpose: Create a CRL builder with borrowed CA and validated update interval.

issuance::CrlBuilder<'a>::set_number

pub fn set_number(&mut self, number: &[u8]) -> QscResult<()>

Purpose: Set a positive CRL sequence number. Duplicate/invalid values fail.

issuance::CrlBuilder<'a>::add_revoked

pub fn add_revoked(&mut self, serial: &[u8], time: &X509Time) -> QscResult<()>

Purpose: Add an issuer-scoped revoked serial and UTC revocation date.

issuance::CrlBuilder<'a>::sign_der

pub unsafe fn sign_der(&self, signer: X509SignCallback, context: *mut c_void, capacity: usize) -> QscResult<Vec<u8>>

Purpose: Produce DER-encoded CRL using the externally owned CA signing key.

Safety: Explicit unsafe Rust boundary; caller must satisfy native pointer/callback validity and lifetime requirements documented in this module.

issuance::verify_crl_der

pub fn verify_crl_der(der: &[u8], issuer: &Certificate, at: &X509Time) -> QscResult<()>

Purpose: Verify a DER CRL against a trusted issuer and explicit validation time.

21.12 kdf.rs (19 public functions)

kdf::pbmac1_derive_key

pub fn pbmac1_derive_key(hash: Pbmac1Hash, password: &[u8], salt: &[u8], iterations: u64, key_length: usize) -> QscResult<Vec<u8>>

Purpose: Derives a PBMAC1 key into a new vector.

kdf::pbmac1_derive_key_into

pub fn pbmac1_derive_key_into(hash: Pbmac1Hash, password: &[u8], salt: &[u8], iterations: u64, key: &mut [u8]) -> QscResult<()>

Purpose: Derives a PBMAC1 key into a caller-owned buffer.

kdf::hkdf_sha2_256_prk_size

pub fn hkdf_sha2_256_prk_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding usize.

Implementation: Macro-generated Rust API; resolves to usize. Return status and capacity are checked by the macro implementation.

kdf::hkdf_sha2_384_prk_size

pub fn hkdf_sha2_384_prk_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding usize.

Implementation: Macro-generated Rust API; resolves to usize. Return status and capacity are checked by the macro implementation.

kdf::hkdf_sha2_512_prk_size

pub fn hkdf_sha2_512_prk_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding usize.

Implementation: Macro-generated Rust API; resolves to usize. Return status and capacity are checked by the macro implementation.

kdf::pbmac1_max_key_size

pub fn pbmac1_max_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding usize.

Implementation: Macro-generated Rust API; resolves to usize. Return status and capacity are checked by the macro implementation.

kdf::pbmac1_max_iterations

pub fn pbmac1_max_iterations() -> QscResult<u64>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding u64.

Implementation: Macro-generated Rust API; resolves to u64. Return status and capacity are checked by the macro implementation.

kdf::hkdf_sha2_256_extract

pub fn hkdf_sha2_256_extract(key: &[u8], salt: &[u8]) -> QscResult<Vec<u8>>

Purpose: Macro-generated public hkdf_extract_fns wrapper; refer to the native binding qsc_sys::kdf::qscabi_hkdf_sha2_256_extract.

Implementation: Macro-generated Rust API; resolves to qsc_sys::kdf::qscabi_hkdf_sha2_256_extract. Return status and capacity are checked by the macro implementation.

kdf::hkdf_sha2_256_extract_into

pub fn hkdf_sha2_256_extract_into(key: &[u8], salt: &[u8], prk: &mut [u8]) -> QscResult<()>

Purpose: Macro-generated public hkdf_extract_fns wrapper; refer to the native binding qsc_sys::kdf::qscabi_hkdf_sha2_256_extract.

Implementation: Macro-generated Rust API; resolves to qsc_sys::kdf::qscabi_hkdf_sha2_256_extract. Return status and capacity are checked by the macro implementation.

kdf::hkdf_sha2_384_extract

pub fn hkdf_sha2_384_extract(key: &[u8], salt: &[u8]) -> QscResult<Vec<u8>>

Purpose: Macro-generated public hkdf_extract_fns wrapper; refer to the native binding qsc_sys::kdf::qscabi_hkdf_sha2_384_extract.

Implementation: Macro-generated Rust API; resolves to qsc_sys::kdf::qscabi_hkdf_sha2_384_extract. Return status and capacity are checked by the macro implementation.

kdf::hkdf_sha2_384_extract_into

pub fn hkdf_sha2_384_extract_into(key: &[u8], salt: &[u8], prk: &mut [u8]) -> QscResult<()>

Purpose: Macro-generated public hkdf_extract_fns wrapper; refer to the native binding qsc_sys::kdf::qscabi_hkdf_sha2_384_extract.

Implementation: Macro-generated Rust API; resolves to qsc_sys::kdf::qscabi_hkdf_sha2_384_extract. Return status and capacity are checked by the macro implementation.

kdf::hkdf_sha2_512_extract

pub fn hkdf_sha2_512_extract(key: &[u8], salt: &[u8]) -> QscResult<Vec<u8>>

Purpose: Macro-generated public hkdf_extract_fns wrapper; refer to the native binding qsc_sys::kdf::qscabi_hkdf_sha2_512_extract.

Implementation: Macro-generated Rust API; resolves to qsc_sys::kdf::qscabi_hkdf_sha2_512_extract. Return status and capacity are checked by the macro implementation.

kdf::hkdf_sha2_512_extract_into

pub fn hkdf_sha2_512_extract_into(key: &[u8], salt: &[u8], prk: &mut [u8]) -> QscResult<()>

Purpose: Macro-generated public hkdf_extract_fns wrapper; refer to the native binding qsc_sys::kdf::qscabi_hkdf_sha2_512_extract.

Implementation: Macro-generated Rust API; resolves to qsc_sys::kdf::qscabi_hkdf_sha2_512_extract. Return status and capacity are checked by the macro implementation.

kdf::hkdf_sha2_256_expand

pub fn hkdf_sha2_256_expand(prk: &[u8], info: &[u8], output_length: usize) -> QscResult<Vec<u8>>

Purpose: Macro-generated public hkdf_expand_fns wrapper; refer to the native binding qsc_sys::kdf::qscabi_hkdf_sha2_256_expand.

Implementation: Macro-generated Rust API; resolves to qsc_sys::kdf::qscabi_hkdf_sha2_256_expand. Return status and capacity are checked by the macro implementation.

kdf::hkdf_sha2_256_expand_into

pub fn hkdf_sha2_256_expand_into(prk: &[u8], info: &[u8], output: &mut [u8]) -> QscResult<()>

Purpose: Macro-generated public hkdf_expand_fns wrapper; refer to the native binding qsc_sys::kdf::qscabi_hkdf_sha2_256_expand.

Implementation: Macro-generated Rust API; resolves to qsc_sys::kdf::qscabi_hkdf_sha2_256_expand. Return status and capacity are checked by the macro implementation.

kdf::hkdf_sha2_384_expand

pub fn hkdf_sha2_384_expand(prk: &[u8], info: &[u8], output_length: usize) -> QscResult<Vec<u8>>

Purpose: Macro-generated public hkdf_expand_fns wrapper; refer to the native binding qsc_sys::kdf::qscabi_hkdf_sha2_384_expand.

Implementation: Macro-generated Rust API; resolves to qsc_sys::kdf::qscabi_hkdf_sha2_384_expand. Return status and capacity are checked by the macro implementation.

kdf::hkdf_sha2_384_expand_into

pub fn hkdf_sha2_384_expand_into(prk: &[u8], info: &[u8], output: &mut [u8]) -> QscResult<()>

Purpose: Macro-generated public hkdf_expand_fns wrapper; refer to the native binding qsc_sys::kdf::qscabi_hkdf_sha2_384_expand.

Implementation: Macro-generated Rust API; resolves to qsc_sys::kdf::qscabi_hkdf_sha2_384_expand. Return status and capacity are checked by the macro implementation.

kdf::hkdf_sha2_512_expand

pub fn hkdf_sha2_512_expand(prk: &[u8], info: &[u8], output_length: usize) -> QscResult<Vec<u8>>

Purpose: Macro-generated public hkdf_expand_fns wrapper; refer to the native binding qsc_sys::kdf::qscabi_hkdf_sha2_512_expand.

Implementation: Macro-generated Rust API; resolves to qsc_sys::kdf::qscabi_hkdf_sha2_512_expand. Return status and capacity are checked by the macro implementation.

kdf::hkdf_sha2_512_expand_into

pub fn hkdf_sha2_512_expand_into(prk: &[u8], info: &[u8], output: &mut [u8]) -> QscResult<()>

Purpose: Macro-generated public hkdf_expand_fns wrapper; refer to the native binding qsc_sys::kdf::qscabi_hkdf_sha2_512_expand.

Implementation: Macro-generated Rust API; resolves to qsc_sys::kdf::qscabi_hkdf_sha2_512_expand. Return status and capacity are checked by the macro implementation.

21.13 kem.rs (21 public functions)

kem::mlkem::generate_keypair

pub fn generate_keypair() -> QscResult<KemKeyPair>

Purpose: Generates an ML-KEM key pair.

kem::mlkem::encapsulate

pub fn encapsulate(public_key: &[u8]) -> QscResult<KemEncapsulation>

Purpose: Encapsulates a shared secret to an ML-KEM public key.

kem::mlkem::decapsulate

pub fn decapsulate(ciphertext: &[u8], private_key: &[u8]) -> QscResult<SecretBytes>

Purpose: Decapsulates an ML-KEM ciphertext with the matching private key.

kem::hqc::generate_keypair

pub fn generate_keypair() -> QscResult<KemKeyPair>

Purpose: Generates an HQC key pair.

kem::hqc::encapsulate

pub fn encapsulate(public_key: &[u8]) -> QscResult<KemEncapsulation>

Purpose: Encapsulates a shared secret to an HQC public key.

kem::hqc::decapsulate

pub fn decapsulate(ciphertext: &[u8], private_key: &[u8]) -> QscResult<SecretBytes>

Purpose: Decapsulates an HQC ciphertext with the matching private key.

kem::mceliece::generate_keypair

pub fn generate_keypair() -> QscResult<KemKeyPair>

Purpose: Generates a Classic McEliece key pair.

kem::mceliece::encapsulate

pub fn encapsulate(public_key: &[u8]) -> QscResult<KemEncapsulation>

Purpose: Encapsulates a shared secret to a Classic McEliece public key.

kem::mceliece::decapsulate

pub fn decapsulate(ciphertext: &[u8], private_key: &[u8]) -> QscResult<SecretBytes>

Purpose: Decapsulates a Classic McEliece ciphertext with the matching private key.

kem::mlkem::public_key_size

pub fn public_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::pqc::qscabi_mlkem_public_key_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::pqc::qscabi_mlkem_public_key_size. Return status and capacity are checked by the macro implementation.

kem::mlkem::private_key_size

pub fn private_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::pqc::qscabi_mlkem_private_key_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::pqc::qscabi_mlkem_private_key_size. Return status and capacity are checked by the macro implementation.

kem::mlkem::ciphertext_size

pub fn ciphertext_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::pqc::qscabi_mlkem_ciphertext_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::pqc::qscabi_mlkem_ciphertext_size. Return status and capacity are checked by the macro implementation.

kem::mlkem::shared_secret_size

pub fn shared_secret_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::pqc::qscabi_mlkem_shared_secret_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::pqc::qscabi_mlkem_shared_secret_size. Return status and capacity are checked by the macro implementation.

kem::hqc::public_key_size

pub fn public_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::pqc::qscabi_hqc_public_key_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::pqc::qscabi_hqc_public_key_size. Return status and capacity are checked by the macro implementation.

kem::hqc::private_key_size

pub fn private_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::pqc::qscabi_hqc_private_key_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::pqc::qscabi_hqc_private_key_size. Return status and capacity are checked by the macro implementation.

kem::hqc::ciphertext_size

pub fn ciphertext_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::pqc::qscabi_hqc_ciphertext_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::pqc::qscabi_hqc_ciphertext_size. Return status and capacity are checked by the macro implementation.

kem::hqc::shared_secret_size

pub fn shared_secret_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::pqc::qscabi_hqc_shared_secret_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::pqc::qscabi_hqc_shared_secret_size. Return status and capacity are checked by the macro implementation.

kem::mceliece::public_key_size

pub fn public_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::pqc::qscabi_mceliece_public_key_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::pqc::qscabi_mceliece_public_key_size. Return status and capacity are checked by the macro implementation.

kem::mceliece::private_key_size

pub fn private_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::pqc::qscabi_mceliece_private_key_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::pqc::qscabi_mceliece_private_key_size. Return status and capacity are checked by the macro implementation.

kem::mceliece::ciphertext_size

pub fn ciphertext_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::pqc::qscabi_mceliece_ciphertext_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::pqc::qscabi_mceliece_ciphertext_size. Return status and capacity are checked by the macro implementation.

kem::mceliece::shared_secret_size

pub fn shared_secret_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::pqc::qscabi_mceliece_shared_secret_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::pqc::qscabi_mceliece_shared_secret_size. Return status and capacity are checked by the macro implementation.

21.14 lib.rs (2 public functions)

lib::native_available

pub fn native_available() -> bool

Purpose: Returns true when the native QSCABI runtime is available to the process.

lib::native_load_error

pub fn native_load_error() -> Option<String>

Purpose: Returns the current native-loader diagnostic if QSCABI is unavailable.

21.15 mac.rs (38 public functions)

mac::kmac_128_default

pub fn kmac_128_default(message: &[u8], key: &[u8], custom: &[u8]) -> QscResult<Vec<u8>>

Purpose: Computes a default-length KMAC-128 code matching the C# wrapper default.

mac::kmac_256_default

pub fn kmac_256_default(message: &[u8], key: &[u8], custom: &[u8]) -> QscResult<Vec<u8>>

Purpose: Computes a default-length KMAC-256 code matching the C# wrapper default.

mac::kmac_512_default

pub fn kmac_512_default(message: &[u8], key: &[u8], custom: &[u8]) -> QscResult<Vec<u8>>

Purpose: Computes a default-length KMAC-512 code matching the C# wrapper default.

mac::pbmac1_mac_size

pub fn pbmac1_mac_size(hash: Pbmac1Hash) -> QscResult<usize>

Purpose: Returns the PBMAC1 MAC size for the selected profile.

mac::poly1305

pub fn poly1305(message: &[u8], key: &[u8]) -> QscResult<Vec<u8>>

Purpose: Computes Poly1305.

mac::poly1305_into

pub fn poly1305_into(message: &[u8], key: &[u8], mac: &mut [u8]) -> QscResult<()>

Purpose: Computes Poly1305 into a caller-owned buffer.

mac::poly1305_verify

pub fn poly1305_verify(code: &[u8], message: &[u8], key: &[u8]) -> QscResult<bool>

Purpose: Verifies a Poly1305 code. Verification failure returns `Ok(false)`.

mac::qmac

pub fn qmac(message: &[u8], key: &[u8], nonce: &[u8], info: &[u8]) -> QscResult<Vec<u8>>

Purpose: Computes QMAC.

mac::qmac_into

pub fn qmac_into(message: &[u8], key: &[u8], nonce: &[u8], info: &[u8], mac: &mut [u8]) -> QscResult<()>

Purpose: Computes QMAC into a caller-owned buffer.

mac::pbmac1

pub fn pbmac1(hash: Pbmac1Hash, password: &[u8], salt: &[u8], iterations: u64, key_length: usize, message: &[u8]) -> QscResult<Vec<u8>>

Purpose: Computes PBMAC1.

mac::pbmac1_into

pub fn pbmac1_into(hash: Pbmac1Hash, password: &[u8], salt: &[u8], iterations: u64, key_length: usize, message: &[u8], mac: &mut [u8]) -> QscResult<()>

Purpose: Computes PBMAC1 into a caller-owned buffer.

mac::pbmac1_verify

pub fn pbmac1_verify(code: &[u8], hash: Pbmac1Hash, password: &[u8], salt: &[u8], iterations: u64, key_length: usize, message: &[u8]) -> QscResult<bool>

Purpose: Verifies PBMAC1. Verification failure returns `Ok(false)`.

mac::hmac_sha2_256_mac_size

pub fn hmac_sha2_256_mac_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::mac::qscabi_hmac_sha2_256_mac_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::mac::qscabi_hmac_sha2_256_mac_size. Return status and capacity are checked by the macro implementation.

mac::hmac_sha2_384_mac_size

pub fn hmac_sha2_384_mac_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::mac::qscabi_hmac_sha2_384_mac_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::mac::qscabi_hmac_sha2_384_mac_size. Return status and capacity are checked by the macro implementation.

mac::hmac_sha2_512_mac_size

pub fn hmac_sha2_512_mac_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::mac::qscabi_hmac_sha2_512_mac_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::mac::qscabi_hmac_sha2_512_mac_size. Return status and capacity are checked by the macro implementation.

mac::poly1305_mac_size

pub fn poly1305_mac_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::mac::qscabi_poly1305_mac_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::mac::qscabi_poly1305_mac_size. Return status and capacity are checked by the macro implementation.

mac::poly1305_key_size

pub fn poly1305_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::mac::qscabi_poly1305_key_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::mac::qscabi_poly1305_key_size. Return status and capacity are checked by the macro implementation.

mac::qmac_mac_size

pub fn qmac_mac_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::mac::qscabi_qmac_mac_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::mac::qscabi_qmac_mac_size. Return status and capacity are checked by the macro implementation.

mac::qmac_key_size

pub fn qmac_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::mac::qscabi_qmac_key_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::mac::qscabi_qmac_key_size. Return status and capacity are checked by the macro implementation.

mac::qmac_nonce_size

pub fn qmac_nonce_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::mac::qscabi_qmac_nonce_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::mac::qscabi_qmac_nonce_size. Return status and capacity are checked by the macro implementation.

mac::hmac_sha2_256

pub fn hmac_sha2_256(message: &[u8], key: &[u8]) -> QscResult<Vec<u8>>

Purpose: Macro-generated public hmac_fns wrapper; refer to the native binding qsc_sys::mac::qscabi_hmac_sha2_256_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::mac::qscabi_hmac_sha2_256_compute. Return status and capacity are checked by the macro implementation.

mac::hmac_sha2_256_into

pub fn hmac_sha2_256_into(message: &[u8], key: &[u8], mac: &mut [u8]) -> QscResult<()>

Purpose: Macro-generated public hmac_fns wrapper; refer to the native binding qsc_sys::mac::qscabi_hmac_sha2_256_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::mac::qscabi_hmac_sha2_256_compute. Return status and capacity are checked by the macro implementation.

mac::hmac_sha2_384

pub fn hmac_sha2_384(message: &[u8], key: &[u8]) -> QscResult<Vec<u8>>

Purpose: Macro-generated public hmac_fns wrapper; refer to the native binding qsc_sys::mac::qscabi_hmac_sha2_384_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::mac::qscabi_hmac_sha2_384_compute. Return status and capacity are checked by the macro implementation.

mac::hmac_sha2_384_into

pub fn hmac_sha2_384_into(message: &[u8], key: &[u8], mac: &mut [u8]) -> QscResult<()>

Purpose: Macro-generated public hmac_fns wrapper; refer to the native binding qsc_sys::mac::qscabi_hmac_sha2_384_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::mac::qscabi_hmac_sha2_384_compute. Return status and capacity are checked by the macro implementation.

mac::hmac_sha2_512

pub fn hmac_sha2_512(message: &[u8], key: &[u8]) -> QscResult<Vec<u8>>

Purpose: Macro-generated public hmac_fns wrapper; refer to the native binding qsc_sys::mac::qscabi_hmac_sha2_512_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::mac::qscabi_hmac_sha2_512_compute. Return status and capacity are checked by the macro implementation.

mac::hmac_sha2_512_into

pub fn hmac_sha2_512_into(message: &[u8], key: &[u8], mac: &mut [u8]) -> QscResult<()>

Purpose: Macro-generated public hmac_fns wrapper; refer to the native binding qsc_sys::mac::qscabi_hmac_sha2_512_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::mac::qscabi_hmac_sha2_512_compute. Return status and capacity are checked by the macro implementation.

mac::kmac_128

pub fn kmac_128(message: &[u8], key: &[u8], custom: &[u8], output_length: usize) -> QscResult<Vec<u8>>

Purpose: Macro-generated public kmac_fns wrapper; refer to the native binding qsc_sys::mac::qscabi_kmac_128_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::mac::qscabi_kmac_128_compute. Return status and capacity are checked by the macro implementation.

mac::kmac_128_into

pub fn kmac_128_into(message: &[u8], key: &[u8], custom: &[u8], output: &mut [u8]) -> QscResult<()>

Purpose: Macro-generated public kmac_fns wrapper; refer to the native binding qsc_sys::mac::qscabi_kmac_128_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::mac::qscabi_kmac_128_compute. Return status and capacity are checked by the macro implementation.

mac::kmac_256

pub fn kmac_256(message: &[u8], key: &[u8], custom: &[u8], output_length: usize) -> QscResult<Vec<u8>>

Purpose: Macro-generated public kmac_fns wrapper; refer to the native binding qsc_sys::mac::qscabi_kmac_256_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::mac::qscabi_kmac_256_compute. Return status and capacity are checked by the macro implementation.

mac::kmac_256_into

pub fn kmac_256_into(message: &[u8], key: &[u8], custom: &[u8], output: &mut [u8]) -> QscResult<()>

Purpose: Macro-generated public kmac_fns wrapper; refer to the native binding qsc_sys::mac::qscabi_kmac_256_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::mac::qscabi_kmac_256_compute. Return status and capacity are checked by the macro implementation.

mac::kmac_512

pub fn kmac_512(message: &[u8], key: &[u8], custom: &[u8], output_length: usize) -> QscResult<Vec<u8>>

Purpose: Macro-generated public kmac_fns wrapper; refer to the native binding qsc_sys::mac::qscabi_kmac_512_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::mac::qscabi_kmac_512_compute. Return status and capacity are checked by the macro implementation.

mac::kmac_512_into

pub fn kmac_512_into(message: &[u8], key: &[u8], custom: &[u8], output: &mut [u8]) -> QscResult<()>

Purpose: Macro-generated public kmac_fns wrapper; refer to the native binding qsc_sys::mac::qscabi_kmac_512_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::mac::qscabi_kmac_512_compute. Return status and capacity are checked by the macro implementation.

mac::kmac_xof_128

pub fn kmac_xof_128(message: &[u8], key: &[u8], custom: &[u8], output_length: usize) -> QscResult<Vec<u8>>

Purpose: Macro-generated public kmac_fns wrapper; refer to the native binding qsc_sys::mac::qscabi_kmac_xof_128_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::mac::qscabi_kmac_xof_128_compute. Return status and capacity are checked by the macro implementation.

mac::kmac_xof_128_into

pub fn kmac_xof_128_into(message: &[u8], key: &[u8], custom: &[u8], output: &mut [u8]) -> QscResult<()>

Purpose: Macro-generated public kmac_fns wrapper; refer to the native binding qsc_sys::mac::qscabi_kmac_xof_128_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::mac::qscabi_kmac_xof_128_compute. Return status and capacity are checked by the macro implementation.

mac::kmac_xof_256

pub fn kmac_xof_256(message: &[u8], key: &[u8], custom: &[u8], output_length: usize) -> QscResult<Vec<u8>>

Purpose: Macro-generated public kmac_fns wrapper; refer to the native binding qsc_sys::mac::qscabi_kmac_xof_256_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::mac::qscabi_kmac_xof_256_compute. Return status and capacity are checked by the macro implementation.

mac::kmac_xof_256_into

pub fn kmac_xof_256_into(message: &[u8], key: &[u8], custom: &[u8], output: &mut [u8]) -> QscResult<()>

Purpose: Macro-generated public kmac_fns wrapper; refer to the native binding qsc_sys::mac::qscabi_kmac_xof_256_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::mac::qscabi_kmac_xof_256_compute. Return status and capacity are checked by the macro implementation.

mac::kmac_xof_512

pub fn kmac_xof_512(message: &[u8], key: &[u8], custom: &[u8], output_length: usize) -> QscResult<Vec<u8>>

Purpose: Macro-generated public kmac_fns wrapper; refer to the native binding qsc_sys::mac::qscabi_kmac_xof_512_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::mac::qscabi_kmac_xof_512_compute. Return status and capacity are checked by the macro implementation.

mac::kmac_xof_512_into

pub fn kmac_xof_512_into(message: &[u8], key: &[u8], custom: &[u8], output: &mut [u8]) -> QscResult<()>

Purpose: Macro-generated public kmac_fns wrapper; refer to the native binding qsc_sys::mac::qscabi_kmac_xof_512_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::mac::qscabi_kmac_xof_512_compute. Return status and capacity are checked by the macro implementation.

21.16 memory.rs (2 public functions)

memory::clear

pub fn clear(data: &mut [u8]) -> QscResult<()>

Purpose: Clears a caller-owned mutable byte slice using QSCABI.

memory::are_equal

pub fn are_equal(left: &[u8], right: &[u8]) -> QscResult<bool>

Purpose: Compares two byte slices using the QSCABI memory-comparison helper.

21.17 secret.rs (6 public functions)

secret::SecretBytes::new

pub fn new(bytes: Vec<u8>) -> Self

Purpose: Creates a new secret byte container from an owned vector.

secret::SecretBytes::as_slice

pub fn as_slice(&self) -> &[u8]

Purpose: Returns the secret bytes as a borrowed slice.

secret::SecretBytes::as_mut_slice

pub fn as_mut_slice(&mut self) -> &mut [u8]

Purpose: Returns a mutable view of the owned secret buffer for a native call.

secret::SecretBytes::len

pub fn len(&self) -> usize

Purpose: Returns the secret length in bytes.

secret::SecretBytes::is_empty

pub fn is_empty(&self) -> bool

Purpose: Returns true when the secret is empty.

secret::SecretBytes::into_bytes

pub fn into_bytes(mut self) -> Vec<u8>

Purpose: Consumes the container and returns the owned bytes.

21.18 signature.rs (41 public functions)

signature::mldsa::generate_keypair

pub fn generate_keypair() -> QscResult<SignatureKeyPair>

Purpose: Generates an ML-DSA key pair.

signature::mldsa::sign

pub fn sign(message: &[u8], private_key: &[u8]) -> QscResult<Vec<u8>>

Purpose: Signs a message and returns the QSCABI signed-message format.

signature::mldsa::verify

pub fn verify(signed_message: &[u8], public_key: &[u8]) -> QscResult<Vec<u8>>

Purpose: Verifies a signed message and returns the recovered message on success.

signature::falcon::generate_keypair

pub fn generate_keypair() -> QscResult<SignatureKeyPair>

Purpose: Generates a Falcon key pair.

signature::falcon::sign

pub fn sign(message: &[u8], private_key: &[u8]) -> QscResult<Vec<u8>>

Purpose: Signs a message and returns the QSCABI signed-message format.

signature::falcon::verify

pub fn verify(signed_message: &[u8], public_key: &[u8]) -> QscResult<Vec<u8>>

Purpose: Verifies a signed message and returns the recovered message on success.

signature::slhdsa::generate_keypair

pub fn generate_keypair() -> QscResult<SignatureKeyPair>

Purpose: Generates an SLH-DSA key pair.

signature::slhdsa::sign

pub fn sign(message: &[u8], private_key: &[u8]) -> QscResult<Vec<u8>>

Purpose: Signs a message and returns the QSCABI signed-message format.

signature::slhdsa::verify

pub fn verify(signed_message: &[u8], public_key: &[u8]) -> QscResult<Vec<u8>>

Purpose: Verifies a signed message and returns the recovered message on success.

signature::ecdsa::generate_keypair

pub fn generate_keypair() -> QscResult<SignatureKeyPair>

Purpose: Generates an ECDSA key pair.

signature::ecdsa::generate_seeded_keypair

pub fn generate_seeded_keypair(seed: &[u8]) -> QscResult<SignatureKeyPair>

Purpose: Generates a deterministic ECDSA key pair from a seed.

signature::ecdsa::public_from_private

pub fn public_from_private(private_key: &[u8]) -> QscResult<Vec<u8>>

Purpose: Derives the ECDSA public key from a private key.

signature::ecdsa::sign

pub fn sign(message: &[u8], private_key: &[u8]) -> QscResult<Vec<u8>>

Purpose: Signs a message and returns the QSCABI signed-message format.

signature::ecdsa::verify

pub fn verify(signed_message: &[u8], public_key: &[u8]) -> QscResult<Vec<u8>>

Purpose: Verifies a signed message and returns the recovered message on success.

signature::eddsa::generate_keypair

pub fn generate_keypair() -> QscResult<SignatureKeyPair>

Purpose: Generates an EdDSA key pair.

signature::eddsa::generate_seeded_keypair

pub fn generate_seeded_keypair(seed: &[u8]) -> QscResult<SignatureKeyPair>

Purpose: Generates a deterministic EdDSA key pair from a seed.

signature::eddsa::public_from_private

pub fn public_from_private(private_key: &[u8]) -> QscResult<Vec<u8>>

Purpose: Derives the EdDSA public key from a private key.

signature::eddsa::sign

pub fn sign(message: &[u8], private_key: &[u8]) -> QscResult<Vec<u8>>

Purpose: Signs a message and returns the QSCABI signed-message format.

signature::eddsa::verify

pub fn verify(signed_message: &[u8], public_key: &[u8]) -> QscResult<Vec<u8>>

Purpose: Verifies a signed message and returns the recovered message on success.

signature::mldsa::public_key_size

pub fn public_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::pqc::qscabi_mldsa_public_key_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::pqc::qscabi_mldsa_public_key_size. Return status and capacity are checked by the macro implementation.

signature::mldsa::private_key_size

pub fn private_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::pqc::qscabi_mldsa_private_key_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::pqc::qscabi_mldsa_private_key_size. Return status and capacity are checked by the macro implementation.

signature::mldsa::signature_size

pub fn signature_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::pqc::qscabi_mldsa_signature_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::pqc::qscabi_mldsa_signature_size. Return status and capacity are checked by the macro implementation.

signature::mldsa::signed_message_size

pub fn signed_message_size(message_length: usize) -> QscResult<usize>

Purpose: Macro-generated public signed_size_fn wrapper; refer to the native binding qsc_sys::pqc::qscabi_mldsa_signed_message_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::pqc::qscabi_mldsa_signed_message_size. Return status and capacity are checked by the macro implementation.

signature::falcon::public_key_size

pub fn public_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::pqc::qscabi_falcon_public_key_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::pqc::qscabi_falcon_public_key_size. Return status and capacity are checked by the macro implementation.

signature::falcon::private_key_size

pub fn private_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::pqc::qscabi_falcon_private_key_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::pqc::qscabi_falcon_private_key_size. Return status and capacity are checked by the macro implementation.

signature::falcon::signature_size

pub fn signature_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::pqc::qscabi_falcon_signature_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::pqc::qscabi_falcon_signature_size. Return status and capacity are checked by the macro implementation.

signature::falcon::signed_message_size

pub fn signed_message_size(message_length: usize) -> QscResult<usize>

Purpose: Macro-generated public signed_size_fn wrapper; refer to the native binding qsc_sys::pqc::qscabi_falcon_signed_message_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::pqc::qscabi_falcon_signed_message_size. Return status and capacity are checked by the macro implementation.

signature::slhdsa::public_key_size

pub fn public_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::pqc::qscabi_slhdsa_public_key_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::pqc::qscabi_slhdsa_public_key_size. Return status and capacity are checked by the macro implementation.

signature::slhdsa::private_key_size

pub fn private_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::pqc::qscabi_slhdsa_private_key_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::pqc::qscabi_slhdsa_private_key_size. Return status and capacity are checked by the macro implementation.

signature::slhdsa::signature_size

pub fn signature_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::pqc::qscabi_slhdsa_signature_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::pqc::qscabi_slhdsa_signature_size. Return status and capacity are checked by the macro implementation.

signature::slhdsa::signed_message_size

pub fn signed_message_size(message_length: usize) -> QscResult<usize>

Purpose: Macro-generated public signed_size_fn wrapper; refer to the native binding qsc_sys::pqc::qscabi_slhdsa_signed_message_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::pqc::qscabi_slhdsa_signed_message_size. Return status and capacity are checked by the macro implementation.

signature::ecdsa::public_key_size

pub fn public_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::asymmetric::qscabi_ecdsa_public_key_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::asymmetric::qscabi_ecdsa_public_key_size. Return status and capacity are checked by the macro implementation.

signature::ecdsa::private_key_size

pub fn private_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::asymmetric::qscabi_ecdsa_private_key_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::asymmetric::qscabi_ecdsa_private_key_size. Return status and capacity are checked by the macro implementation.

signature::ecdsa::signature_size

pub fn signature_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::asymmetric::qscabi_ecdsa_signature_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::asymmetric::qscabi_ecdsa_signature_size. Return status and capacity are checked by the macro implementation.

signature::ecdsa::seed_size

pub fn seed_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::asymmetric::qscabi_ecdsa_seed_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::asymmetric::qscabi_ecdsa_seed_size. Return status and capacity are checked by the macro implementation.

signature::ecdsa::signed_message_size

pub fn signed_message_size(message_length: usize) -> QscResult<usize>

Purpose: Macro-generated public signed_size_fn wrapper; refer to the native binding qsc_sys::asymmetric::qscabi_ecdsa_signed_message_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::asymmetric::qscabi_ecdsa_signed_message_size. Return status and capacity are checked by the macro implementation.

signature::eddsa::public_key_size

pub fn public_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::asymmetric::qscabi_eddsa_public_key_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::asymmetric::qscabi_eddsa_public_key_size. Return status and capacity are checked by the macro implementation.

signature::eddsa::private_key_size

pub fn private_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::asymmetric::qscabi_eddsa_private_key_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::asymmetric::qscabi_eddsa_private_key_size. Return status and capacity are checked by the macro implementation.

signature::eddsa::signature_size

pub fn signature_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::asymmetric::qscabi_eddsa_signature_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::asymmetric::qscabi_eddsa_signature_size. Return status and capacity are checked by the macro implementation.

signature::eddsa::seed_size

pub fn seed_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::asymmetric::qscabi_eddsa_seed_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::asymmetric::qscabi_eddsa_seed_size. Return status and capacity are checked by the macro implementation.

signature::eddsa::signed_message_size

pub fn signed_message_size(message_length: usize) -> QscResult<usize>

Purpose: Macro-generated public signed_size_fn wrapper; refer to the native binding qsc_sys::asymmetric::qscabi_eddsa_signed_message_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::asymmetric::qscabi_eddsa_signed_message_size. Return status and capacity are checked by the macro implementation.

21.19 status.rs (1 public functions)

status::native_status_string

pub fn native_status_string(status: QscStatus) -> crate::QscResult<String>

Purpose: Returns the native QSCABI status string for a status value.

21.20 streaming.rs (13 public functions)

streaming::Digest::new

pub fn new(algorithm: DigestAlgorithm) -> QscResult<Self>

Purpose: Allocate a new native digest context.

streaming::Digest::update

pub fn update(&mut self, data: &[u8]) -> QscResult<()>

Purpose: Absorb borrowed message bytes, including an empty chunk.

streaming::Digest::finalize

pub fn finalize(&mut self) -> QscResult<Vec<u8>>

Purpose: Produce the final digest and prevent further updates or finalizations.

streaming::Xof::new

pub fn new(algorithm: XofAlgorithm) -> QscResult<Self>

Purpose: Initialize a SHAKE sponge for absorbing message bytes.

streaming::Xof::absorb

pub fn absorb(&mut self, input: &[u8]) -> QscResult<()>

Purpose: Absorb additional borrowed message bytes before finalization.

streaming::Xof::finalize

pub fn finalize(&mut self) -> QscResult<()>

Purpose: Switch from absorption to squeezing; may occur exactly once.

streaming::Xof::squeeze

pub fn squeeze(&mut self, length: usize) -> QscResult<Vec<u8>>

Purpose: Obtain the next `length` bytes without resetting the sponge position.

streaming::cshake

pub fn cshake(algorithm: XofAlgorithm, input: &[u8], name: &[u8], custom: &[u8], length: usize) -> QscResult<Vec<u8>>

Purpose: Compute cSHAKE with a function-name and customization domain separation.

streaming::Mac::new

pub fn new(algorithm: MacAlgorithm, key: &[u8], custom: &[u8]) -> QscResult<Self>

Purpose: Create MAC state, copying a borrowed secret key into native memory.

streaming::Mac::update

pub fn update(&mut self, input: &[u8]) -> QscResult<()>

Purpose: Absorb borrowed bytes; updates following finalize are rejected.

streaming::Mac::finalize

pub fn finalize(&mut self, length: usize) -> QscResult<Vec<u8>>

Purpose: Produce an HMAC tag of fixed length or a variable-length KMAC tag.

streaming::Ctr::new

pub fn new(algorithm: CtrAlgorithm, key: &[u8], nonce: &[u8]) -> QscResult<Self>

Purpose: Create an AES-128/256 CTR context using an application-unique 16-byte nonce.

streaming::Ctr::transform

pub fn transform(&mut self, input: &[u8]) -> QscResult<Vec<u8>>

Purpose: Encrypt or decrypt bytes (same transform) and reject reuse after a partial block.

21.21 symmetric.rs (40 public functions)

symmetric::rcs_key_size

pub fn rcs_key_size(mode: RcsMode) -> QscResult<usize>

Purpose: Returns the RCS key size for the selected mode.

symmetric::rcs_tag_size

pub fn rcs_tag_size(mode: RcsMode) -> QscResult<usize>

Purpose: Returns the RCS authentication-tag size for the selected mode.

symmetric::aes256_ctr_transform

pub fn aes256_ctr_transform(input: &[u8], key: &[u8], nonce: &[u8]) -> QscResult<Vec<u8>>

Purpose: Transforms data with AES-256-CTR using big-endian counter semantics.

symmetric::aes256_ctr_transform_with_counter

pub fn aes256_ctr_transform_with_counter(input: &[u8], key: &[u8], nonce: &[u8], counter_mode: AesCtrCounter,) -> QscResult<Vec<u8>>

Purpose: Transforms data with AES-256-CTR using the selected counter semantics.

symmetric::aes256_ctr_transform_into

pub fn aes256_ctr_transform_into(input: &[u8], key: &[u8], nonce: &[u8], counter_mode: AesCtrCounter, output: &mut [u8],) -> QscResult<()>

Purpose: Transforms AES-256-CTR into a caller-owned output buffer.

symmetric::aes128_gcm_encrypt

pub fn aes128_gcm_encrypt(plaintext: &[u8], key: &[u8], nonce: &[u8], aad: &[u8]) -> QscResult<Vec<u8>>

Purpose: Encrypts plaintext with AES-128-GCM and appends the authentication tag.

symmetric::aes128_gcm_decrypt

pub fn aes128_gcm_decrypt(ciphertext: &[u8], key: &[u8], nonce: &[u8], aad: &[u8]) -> QscResult<Vec<u8>>

Purpose: Decrypts and authenticates AES-128-GCM ciphertext.

symmetric::aes128_gcm_encrypt_into

pub fn aes128_gcm_encrypt_into(plaintext: &[u8], key: &[u8], nonce: &[u8], aad: &[u8], output: &mut [u8],) -> QscResult<usize>

Purpose: Encrypts AES-128-GCM into a caller-owned output buffer.

symmetric::aes128_gcm_decrypt_into

pub fn aes128_gcm_decrypt_into(ciphertext: &[u8], key: &[u8], nonce: &[u8], aad: &[u8], output: &mut [u8],) -> QscResult<usize>

Purpose: Decrypts AES-128-GCM into a caller-owned output buffer.

symmetric::aes256_gcm_encrypt

pub fn aes256_gcm_encrypt(plaintext: &[u8], key: &[u8], nonce: &[u8], aad: &[u8]) -> QscResult<Vec<u8>>

Purpose: Encrypts plaintext with AES-256-GCM and appends the authentication tag.

symmetric::aes256_gcm_decrypt

pub fn aes256_gcm_decrypt(ciphertext: &[u8], key: &[u8], nonce: &[u8], aad: &[u8]) -> QscResult<Vec<u8>>

Purpose: Decrypts and authenticates AES-256-GCM ciphertext.

symmetric::aes256_gcm_encrypt_into

pub fn aes256_gcm_encrypt_into(plaintext: &[u8], key: &[u8], nonce: &[u8], aad: &[u8], output: &mut [u8],) -> QscResult<usize>

Purpose: Encrypts AES-256-GCM into a caller-owned output buffer.

symmetric::aes256_gcm_decrypt_into

pub fn aes256_gcm_decrypt_into(ciphertext: &[u8], key: &[u8], nonce: &[u8], aad: &[u8], output: &mut [u8],) -> QscResult<usize>

Purpose: Decrypts AES-256-GCM into a caller-owned output buffer.

symmetric::chacha20_transform

pub fn chacha20_transform(input: &[u8], key: &[u8], nonce: &[u8]) -> QscResult<Vec<u8>>

Purpose: Transforms data with ChaCha20.

symmetric::chacha20_transform_into

pub fn chacha20_transform_into(input: &[u8], key: &[u8], nonce: &[u8], output: &mut [u8]) -> QscResult<()>

Purpose: Transforms ChaCha20 into a caller-owned output buffer.

symmetric::chacha20_poly1305_encrypt

pub fn chacha20_poly1305_encrypt(plaintext: &[u8], key: &[u8], nonce: &[u8], aad: &[u8]) -> QscResult<Vec<u8>>

Purpose: Encrypts plaintext with ChaCha20-Poly1305 and appends the authentication tag.

symmetric::chacha20_poly1305_decrypt

pub fn chacha20_poly1305_decrypt(ciphertext: &[u8], key: &[u8], nonce: &[u8], aad: &[u8]) -> QscResult<Vec<u8>>

Purpose: Decrypts and authenticates ChaCha20-Poly1305 ciphertext.

symmetric::chacha20_poly1305_encrypt_into

pub fn chacha20_poly1305_encrypt_into(plaintext: &[u8], key: &[u8], nonce: &[u8], aad: &[u8], output: &mut [u8],) -> QscResult<usize>

Purpose: Encrypts ChaCha20-Poly1305 into a caller-owned output buffer.

symmetric::chacha20_poly1305_decrypt_into

pub fn chacha20_poly1305_decrypt_into(ciphertext: &[u8], key: &[u8], nonce: &[u8], aad: &[u8], output: &mut [u8],) -> QscResult<usize>

Purpose: Decrypts ChaCha20-Poly1305 into a caller-owned output buffer.

symmetric::csx_encrypt

pub fn csx_encrypt(plaintext: &[u8], key: &[u8], nonce: &[u8], info: &[u8], aad: &[u8]) -> QscResult<Vec<u8>>

Purpose: Encrypts plaintext with CSX and appends the authentication tag.

symmetric::csx_decrypt

pub fn csx_decrypt(ciphertext: &[u8], key: &[u8], nonce: &[u8], info: &[u8], aad: &[u8]) -> QscResult<Vec<u8>>

Purpose: Decrypts and authenticates CSX ciphertext.

symmetric::csx_encrypt_into

pub fn csx_encrypt_into(plaintext: &[u8], key: &[u8], nonce: &[u8], info: &[u8], aad: &[u8], output: &mut [u8],) -> QscResult<usize>

Purpose: Encrypts CSX into a caller-owned output buffer.

symmetric::csx_decrypt_into

pub fn csx_decrypt_into(ciphertext: &[u8], key: &[u8], nonce: &[u8], info: &[u8], aad: &[u8], output: &mut [u8],) -> QscResult<usize>

Purpose: Decrypts CSX into a caller-owned output buffer.

symmetric::rcs_encrypt

pub fn rcs_encrypt(mode: RcsMode, plaintext: &[u8], key: &[u8], nonce: &[u8], info: &[u8], aad: &[u8]) -> QscResult<Vec<u8>>

Purpose: Encrypts plaintext with RCS and appends the profile-specific authentication tag.

symmetric::rcs_decrypt

pub fn rcs_decrypt(mode: RcsMode, ciphertext: &[u8], key: &[u8], nonce: &[u8], info: &[u8], aad: &[u8]) -> QscResult<Vec<u8>>

Purpose: Decrypts and authenticates RCS ciphertext.

symmetric::rcs_encrypt_into

pub fn rcs_encrypt_into(mode: RcsMode, plaintext: &[u8], key: &[u8], nonce: &[u8], info: &[u8], aad: &[u8], output: &mut [u8],) -> QscResult<usize>

Purpose: Encrypts RCS into a caller-owned output buffer.

symmetric::rcs_decrypt_into

pub fn rcs_decrypt_into(mode: RcsMode, ciphertext: &[u8], key: &[u8], nonce: &[u8], info: &[u8], aad: &[u8], output: &mut [u8],) -> QscResult<usize>

Purpose: Decrypts RCS into a caller-owned output buffer.

symmetric::aes_block_size

pub fn aes_block_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::symmetric::qscabi_aes_block_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::symmetric::qscabi_aes_block_size. Return status and capacity are checked by the macro implementation.

symmetric::aes128_key_size

pub fn aes128_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::symmetric::qscabi_aes128_key_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::symmetric::qscabi_aes128_key_size. Return status and capacity are checked by the macro implementation.

symmetric::aes256_key_size

pub fn aes256_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::symmetric::qscabi_aes256_key_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::symmetric::qscabi_aes256_key_size. Return status and capacity are checked by the macro implementation.

symmetric::aes_gcm_nonce_size

pub fn aes_gcm_nonce_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::symmetric::qscabi_aes_gcm_nonce_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::symmetric::qscabi_aes_gcm_nonce_size. Return status and capacity are checked by the macro implementation.

symmetric::aes_gcm_max_nonce_size

pub fn aes_gcm_max_nonce_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::symmetric::qscabi_aes_gcm_max_nonce_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::symmetric::qscabi_aes_gcm_max_nonce_size. Return status and capacity are checked by the macro implementation.

symmetric::aes_gcm_tag_size

pub fn aes_gcm_tag_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::symmetric::qscabi_aes_gcm_tag_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::symmetric::qscabi_aes_gcm_tag_size. Return status and capacity are checked by the macro implementation.

symmetric::chacha20_key_size

pub fn chacha20_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::symmetric::qscabi_chacha20_key_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::symmetric::qscabi_chacha20_key_size. Return status and capacity are checked by the macro implementation.

symmetric::chacha20_nonce_size

pub fn chacha20_nonce_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::symmetric::qscabi_chacha20_nonce_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::symmetric::qscabi_chacha20_nonce_size. Return status and capacity are checked by the macro implementation.

symmetric::chacha20_poly1305_tag_size

pub fn chacha20_poly1305_tag_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::symmetric::qscabi_chacha20_poly1305_tag_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::symmetric::qscabi_chacha20_poly1305_tag_size. Return status and capacity are checked by the macro implementation.

symmetric::csx_key_size

pub fn csx_key_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::symmetric::qscabi_csx_key_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::symmetric::qscabi_csx_key_size. Return status and capacity are checked by the macro implementation.

symmetric::csx_nonce_size

pub fn csx_nonce_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::symmetric::qscabi_csx_nonce_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::symmetric::qscabi_csx_nonce_size. Return status and capacity are checked by the macro implementation.

symmetric::csx_tag_size

pub fn csx_tag_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::symmetric::qscabi_csx_tag_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::symmetric::qscabi_csx_tag_size. Return status and capacity are checked by the macro implementation.

symmetric::rcs_nonce_size

pub fn rcs_nonce_size() -> QscResult<usize>

Purpose: Macro-generated public size_fn wrapper; refer to the native binding qsc_sys::symmetric::qscabi_rcs_nonce_size.

Implementation: Macro-generated Rust API; resolves to qsc_sys::symmetric::qscabi_rcs_nonce_size. Return status and capacity are checked by the macro implementation.

21.22 tls.rs (45 public functions)

tls::SocketOptions::native_default

pub fn native_default() -> QscResult<Self>

Purpose: Reads the native QSCABI default socket options.

tls::TicketPolicy::native_default

pub fn native_default() -> QscResult<Self>

Purpose: Reads the native QSCABI default session-ticket policy.

tls::TlsContext::create

pub fn create() -> QscResult<Self>

Purpose: Creates a new TLS context handle.

tls::TlsContext::set_policy

pub fn set_policy(&mut self, policy: TlsPolicy) -> QscResult<()>

Purpose: Applies a predefined TLS policy.

tls::TlsContext::set_socket_options

pub fn set_socket_options(&mut self, options: &SocketOptions) -> QscResult<()>

Purpose: Applies socket options to this context.

tls::TlsContext::set_alpn_protocols

pub fn set_alpn_protocols(&mut self, protocols: &[&str], required: bool) -> QscResult<()>

Purpose: Configures ALPN protocol identifiers.

tls::TlsContext::clear_alpn_protocols

pub fn clear_alpn_protocols(&mut self) -> QscResult<()>

Purpose: Clears ALPN protocol configuration.

tls::TlsContext::set_sni_required

pub fn set_sni_required(&mut self, required: bool) -> QscResult<()>

Purpose: Configures whether SNI is required.

tls::TlsContext::set_client_auth

pub fn set_client_auth(&mut self, request_client_auth: bool, require_client_auth: bool) -> QscResult<()>

Purpose: Configures client-certificate request and requirement policy.

tls::TlsContext::set_ticket_policy

pub fn set_ticket_policy(&mut self, policy: &TicketPolicy) -> QscResult<()>

Purpose: Applies a session-ticket policy.

tls::TlsContext::load_trust_anchor_pem

pub fn load_trust_anchor_pem(&mut self, pem: &str, self_signed: bool) -> QscResult<()>

Purpose: Loads a PEM trust anchor into this context.

tls::TlsContext::load_server_identity_pem

pub fn load_server_identity_pem(&mut self, chain_pem: &str, private_key_pem: &str, signature_scheme: SignatureScheme) -> QscResult<()>

Purpose: Loads a PEM certificate chain and private key as server identity material.

tls::TlsContext::set_cipher_suites

pub fn set_cipher_suites(&mut self, suites: &[u16]) -> QscResult<()>

Purpose: Configure TLS cipher-suite identifiers in the application's preferred order.

tls::TlsContext::set_named_groups

pub fn set_named_groups(&mut self, groups: &[u16]) -> QscResult<()>

Purpose: Configure preferred TLS named-group identifiers (unsigned 16-bit IDs).

tls::TlsContext::set_signature_ids

pub fn set_signature_ids(&mut self, schemes: &[u16]) -> QscResult<()>

Purpose: Configure accepted TLS signature-scheme identifiers in priority order.

tls::TlsContext::load_trust_anchor_file

pub fn load_trust_anchor_file(&mut self, path: &std::path::Path, self_signed: bool) -> QscResult<()>

Purpose: Load a trusted PEM certificate from an existing caller-controlled path.

tls::TlsContext::load_trust_anchor_bundle_file

pub fn load_trust_anchor_bundle_file(&mut self, path: &std::path::Path, self_signed: bool) -> QscResult<()>

Purpose: Load a trusted multi-anchor PEM bundle from an existing path.

tls::TlsContext::load_crl_file

pub fn load_crl_file(&mut self, path: &std::path::Path) -> QscResult<()>

Purpose: Load a DER or PEM CRL used by native TLS verification policy.

tls::TlsContext::load_server_identity_files

pub fn load_server_identity_files(&mut self, chain_path: &std::path::Path, key_path: &std::path::Path, scheme: SignatureScheme) -> QscResult<()>

Purpose: Load server-chain and private-key files with an explicit TLS signature scheme.

tls::TlsContext::load_client_identity_files

pub fn load_client_identity_files(&mut self, chain_path: &std::path::Path, key_path: &std::path::Path, scheme: SignatureScheme) -> QscResult<()>

Purpose: Load client-chain and private-key files with an explicit signature scheme.

tls::TlsContext::add_server_identity_files

pub fn add_server_identity_files(&mut self, hostname: &str, chain_path: &std::path::Path, key_path: &std::path::Path, scheme: SignatureScheme) -> QscResult<()>

Purpose: Add a distinct SNI hostname/identity without replacing other server keys.

tls::TlsConnection::create

pub fn create() -> QscResult<Self>

Purpose: Creates an idle TLS connection handle.

tls::TlsConnection::connect_host

pub fn connect_host(&mut self, context: &TlsContext, hostname: &str, service: &str) -> QscResult<()>

Purpose: Connects this connection to a host using a configured client context.

tls::TlsConnection::send

pub fn send(&mut self, input: &[u8]) -> QscResult<usize>

Purpose: Sends application data over the TLS connection.

tls::TlsConnection::receive

pub fn receive(&mut self, output: &mut [u8]) -> QscResult<usize>

Purpose: Receives application data into a caller-owned buffer.

tls::TlsConnection::send_frame

pub fn send_frame(&mut self, input: &[u8]) -> QscResult<()>

Purpose: Sends one application frame.

tls::TlsConnection::receive_frame

pub fn receive_frame(&mut self, output: &mut [u8]) -> QscResult<usize>

Purpose: Receives one application frame into a caller-owned buffer.

tls::TlsConnection::shutdown

pub fn shutdown(&mut self) -> QscResult<()>

Purpose: Initiates a TLS shutdown.

tls::TlsConnection::cancel

pub fn cancel(&mut self) -> QscResult<()>

Purpose: Cancels the connection.

tls::TlsConnection::is_connected

pub fn is_connected(&self) -> QscResult<bool>

Purpose: Returns true when the native connection reports connected state.

tls::TlsConnection::is_handshake_complete

pub fn is_handshake_complete(&self) -> QscResult<bool>

Purpose: Returns true when the TLS handshake is complete.

tls::TlsConnection::selected_alpn

pub fn selected_alpn(&self) -> QscResult<String>

Purpose: Returns the selected ALPN protocol, if one is available.

tls::TlsConnection::peer_info

pub fn peer_info(&self) -> QscResult<PeerInfo>

Purpose: Returns peer information from a completed connection.

tls::TlsListener::create

pub fn create() -> QscResult<Self>

Purpose: Creates a TLS listener handle.

tls::TlsListener::set_socket_options

pub fn set_socket_options(&mut self, options: &SocketOptions) -> QscResult<()>

Purpose: Applies socket options to this listener.

tls::TlsListener::bind

pub fn bind(&mut self, context: &TlsContext, address: &str, port: u16, family: AddressFamily) -> QscResult<()>

Purpose: Binds this listener to an address and port using a configured server context.

tls::TlsListener::accept

pub fn accept(&mut self, connection: &mut TlsConnection) -> QscResult<()>

Purpose: Accepts a connection into an existing idle connection handle.

tls::TlsListener::close

pub fn close(&mut self) -> QscResult<()>

Purpose: Closes the listener endpoint.

tls::TlsServer::create

pub fn create() -> QscResult<Self>

Purpose: Creates a TLS server handle.

tls::TlsServer::configure

pub fn configure(&mut self, context: &TlsContext, address: &str, port: u16, family: AddressFamily) -> QscResult<()>

Purpose: Configures the server endpoint. This does not start the server.

tls::TlsServer::set_max_clients

pub fn set_max_clients(&mut self, max_clients: usize) -> QscResult<()>

Purpose: Sets the maximum number of clients accepted by the server.

tls::TlsServer::start

pub fn start(&mut self) -> QscResult<()>

Purpose: Starts the server in the current native execution model.

tls::TlsServer::start_concurrent

pub fn start_concurrent(&mut self) -> QscResult<()>

Purpose: Starts the server using the native concurrent mode.

tls::TlsServer::stop

pub fn stop(&mut self) -> QscResult<()>

Purpose: Stops the server.

tls::status_string

pub fn status_string(status: crate::QscStatus) -> QscResult<String>

Purpose: Returns a TLS-specific status string from QSCABI.

21.23 version.rs (14 public functions)

version::major

pub fn major() -> QscResult<u32>

Purpose: Returns the QSCABI major version.

version::minor

pub fn minor() -> QscResult<u32>

Purpose: Returns the QSCABI minor version.

version::patch

pub fn patch() -> QscResult<u32>

Purpose: Returns the QSCABI patch version.

version::abi_version_string

pub fn abi_version_string() -> QscResult<String>

Purpose: Returns the QSCABI human-readable version string.

version::qsc_version_string

pub fn qsc_version_string() -> QscResult<String>

Purpose: Returns the linked QSC native implementation version string.

version::build_platform

pub fn build_platform() -> QscResult<String>

Purpose: Returns the native QSCABI build platform string.

version::build_compiler

pub fn build_compiler() -> QscResult<String>

Purpose: Returns the native QSCABI build compiler string.

version::enabled_features

pub fn enabled_features() -> QscResult<String>

Purpose: Returns the QSCABI enabled-features string.

version::info

pub fn info() -> QscResult<VersionInfo>

Purpose: Returns all Stage 1 version/build metadata in one structure.

version::abi_is_compatible

pub fn abi_is_compatible(major: u32, minimum_minor: u32) -> QscResult<bool>

Purpose: Query C's ABI major/minor compatibility check, with a minimum minor version.

version::native_capability_flags

pub fn native_capability_flags() -> QscResult<u64>

Purpose: Return the authoritative native 64-bit feature mask with no truncation.

version::native_build_backend

pub fn native_build_backend() -> QscResult<String>

Purpose: Obtain the native C backend identifier (immutable library-owned string).

version::native_qsc_build_profile

pub fn native_qsc_build_profile() -> QscResult<String>

Purpose: Obtain the compiled QSC profile independently of host CPU features.

version::native_build_info

pub fn native_build_info() -> QscResult<NativeBuildInfo>

Purpose: Query all new native-build facts without changing the legacy `info()` API.

21.24 x509.rs (26 public functions)

x509::Certificate::load_memory

pub fn load_memory(data: &[u8]) -> QscResult<Self>

Purpose: Loads a certificate from a memory buffer accepted by QSCABI.

x509::Certificate::load_pem

pub fn load_pem(pem: &str) -> QscResult<Self>

Purpose: Loads a certificate from PEM-armored text.

x509::Certificate::load_der

pub fn load_der(der: &[u8]) -> QscResult<Self>

Purpose: Loads a certificate from DER bytes.

x509::Certificate::export_der

pub fn export_der(&self) -> QscResult<Vec<u8>>

Purpose: Exports this certificate as DER bytes.

x509::Certificate::export_pem

pub fn export_pem(&self) -> QscResult<String>

Purpose: Exports this certificate as PEM text.

x509::Certificate::subject

pub fn subject(&self) -> QscResult<String>

Purpose: Returns the formatted subject distinguished-name string.

x509::Certificate::issuer

pub fn issuer(&self) -> QscResult<String>

Purpose: Returns the formatted issuer distinguished-name string.

x509::Certificate::subject_common_name

pub fn subject_common_name(&self) -> QscResult<String>

Purpose: Returns the subject common-name string.

x509::Certificate::dns_name_count

pub fn dns_name_count(&self) -> QscResult<usize>

Purpose: Returns the number of DNS subject-alternative-name entries.

x509::Certificate::dns_name

pub fn dns_name(&self, index: usize) -> QscResult<String>

Purpose: Returns a DNS subject-alternative-name entry by zero-based index.

x509::Certificate::matches_hostname

pub fn matches_hostname(&self, hostname: &str) -> QscResult<bool>

Purpose: Returns true when QSCABI accepts the hostname for this certificate.

x509::Certificate::match_hostname_status

pub fn match_hostname_status(&self, hostname: &str) -> QscResult<QscStatus>

Purpose: Returns the raw QSCABI status for a hostname match operation.

x509::Certificate::check_role

pub fn check_role(&self, role: CertificateRole, hostname: Option<&str>) -> QscResult<bool>

Purpose: Returns true when QSCABI accepts this certificate for the requested role.

x509::Certificate::check_role_status

pub fn check_role_status(&self, role: CertificateRole, hostname: Option<&str>) -> QscResult<QscStatus>

Purpose: Returns the raw QSCABI status for a certificate role check.

x509::Chain::create

pub fn create(certificate_capacity: usize) -> QscResult<Self>

Purpose: Creates an empty certificate chain with the requested certificate capacity.

x509::Chain::load_memory

pub fn load_memory(data: &[u8], certificate_capacity: usize) -> QscResult<Self>

Purpose: Loads a certificate chain from a caller-supplied memory buffer.

x509::Chain::load_pem

pub fn load_pem(pem: &str, certificate_capacity: usize) -> QscResult<Self>

Purpose: Loads a certificate chain from concatenated PEM text.

x509::Chain::count

pub fn count(&self) -> QscResult<usize>

Purpose: Returns the number of parsed certificates currently loaded into the chain.

x509::TrustStore::create

pub fn create(anchor_capacity: usize) -> QscResult<Self>

Purpose: Creates an empty trust store with the requested trust-anchor capacity.

x509::TrustStore::add_anchor_memory

pub fn add_anchor_memory(&mut self, data: &[u8], self_signed: bool) -> QscResult<()>

Purpose: Adds a trust anchor from an encoded memory buffer.

x509::TrustStore::add_anchor_pem

pub fn add_anchor_pem(&mut self, pem: &str, self_signed: bool) -> QscResult<()>

Purpose: Adds a PEM-armored trust anchor.

x509::TrustStore::count

pub fn count(&self) -> QscResult<usize>

Purpose: Returns the number of trust anchors currently loaded into the store.

x509::TrustStore::verify_peer_certificates

pub fn verify_peer_certificates(&self, chain: &Chain, preset: ProfilePreset, hostname: Option<&str>) -> QscResult<bool>

Purpose: Returns true when QSCABI verifies the peer certificate chain against this store.

x509::TrustStore::verify_peer_certificates_status

pub fn verify_peer_certificates_status(&self, chain: &Chain, preset: ProfilePreset, hostname: Option<&str>) -> QscResult<QscStatus>

Purpose: Returns the raw QSCABI status for peer certificate-chain verification.

x509::TrustStore::add_crl_memory

pub fn add_crl_memory(&mut self, der: &[u8]) -> QscResult<()>

Purpose: Load a signed CRL, enforcing the existing trust store's revocation policy.

x509::status_string

pub fn status_string(status: QscStatus) -> QscResult<String>

Purpose: Returns the native X.509-specific diagnostic string for a QSCABI status code.

21.25 xof.rs (6 public functions)

xof::shake_128

pub fn shake_128(input: &[u8], output_length: usize) -> QscResult<Vec<u8>>

Purpose: Macro-generated public xof_fns wrapper; refer to the native binding qsc_sys::xof::qscabi_shake_128_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::xof::qscabi_shake_128_compute. Return status and capacity are checked by the macro implementation.

xof::shake_128_into

pub fn shake_128_into(input: &[u8], output: &mut [u8]) -> QscResult<()>

Purpose: Macro-generated public xof_fns wrapper; refer to the native binding qsc_sys::xof::qscabi_shake_128_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::xof::qscabi_shake_128_compute. Return status and capacity are checked by the macro implementation.

xof::shake_256

pub fn shake_256(input: &[u8], output_length: usize) -> QscResult<Vec<u8>>

Purpose: Macro-generated public xof_fns wrapper; refer to the native binding qsc_sys::xof::qscabi_shake_256_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::xof::qscabi_shake_256_compute. Return status and capacity are checked by the macro implementation.

xof::shake_256_into

pub fn shake_256_into(input: &[u8], output: &mut [u8]) -> QscResult<()>

Purpose: Macro-generated public xof_fns wrapper; refer to the native binding qsc_sys::xof::qscabi_shake_256_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::xof::qscabi_shake_256_compute. Return status and capacity are checked by the macro implementation.

xof::shake_512

pub fn shake_512(input: &[u8], output_length: usize) -> QscResult<Vec<u8>>

Purpose: Macro-generated public xof_fns wrapper; refer to the native binding qsc_sys::xof::qscabi_shake_512_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::xof::qscabi_shake_512_compute. Return status and capacity are checked by the macro implementation.

xof::shake_512_into

pub fn shake_512_into(input: &[u8], output: &mut [u8]) -> QscResult<()>

Purpose: Macro-generated public xof_fns wrapper; refer to the native binding qsc_sys::xof::qscabi_shake_512_compute.

Implementation: Macro-generated Rust API; resolves to qsc_sys::xof::qscabi_shake_512_compute. Return status and capacity are checked by the macro implementation.

22: Low-Level qsc-sys Binding Reference

The qsc-sys crate declares 177 explicit public function wrappers across its low-level FFI modules (excluding loader.rs, which is summarized separately). This is distinct from 252 eager native function-pointer fields plus the optional/additive calls, all of which together implement the frozen 329-name native contract. The signatures below are raw C-mapped Rust operations intended for wrapper maintainers, not general application use.

22.1 abi010.rs (4 declared Rust functions)

qsc_sys::abi010::abi_is_compatible

pub unsafe fn abi_is_compatible(major: u32, minor: u32) -> Result<u32, LoadError>

Source contract: Bind the exact native `qscabi_abi_is_compatible` declaration from `qscabi_version.h`.

qsc_sys::abi010::capability_flags

pub unsafe fn capability_flags() -> Result<u64, LoadError>

Source contract: Bind the exact native `qscabi_capability_flags` declaration from `qscabi_version.h`.

qsc_sys::abi010::build_backend

pub unsafe fn build_backend() -> Result<*const c_char, LoadError>

Source contract: Bind the exact native `qscabi_build_backend` declaration from `qscabi_version.h`.

qsc_sys::abi010::qsc_build_profile

pub unsafe fn qsc_build_profile() -> Result<*const c_char, LoadError>

Source contract: Bind the exact native `qscabi_qsc_build_profile` declaration from `qscabi_version.h`.

22.2 encoding.rs (12 declared Rust functions)

qsc_sys::encoding::qscabi_hex_encoded_size

pub unsafe fn qscabi_hex_encoded_size(input_length: usize) -> Result<usize, crate::loader::LoadError>

Source contract: Calls `qscabi_hex_encoded_size`.

qsc_sys::encoding::qscabi_hex_decoded_size

pub unsafe fn qscabi_hex_decoded_size(output_length: *mut usize, input: *const c_char, input_length: usize) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Calls `qscabi_hex_decoded_size`.

qsc_sys::encoding::qscabi_hex_encode

pub unsafe fn qscabi_hex_encode(output: *mut c_char, output_length: usize, written: *mut usize, input: *const u8, input_length: usize) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Calls `qscabi_hex_encode`.

qsc_sys::encoding::qscabi_hex_decode

pub unsafe fn qscabi_hex_decode(output: *mut u8, output_length: usize, written: *mut usize, input: *const c_char, input_length: usize) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Calls `qscabi_hex_decode`.

qsc_sys::encoding::qscabi_base64_encoded_size

pub unsafe fn qscabi_base64_encoded_size(input_length: usize) -> Result<usize, crate::loader::LoadError>

Source contract: Calls `qscabi_base64_encoded_size`.

qsc_sys::encoding::qscabi_base64_decoded_size

pub unsafe fn qscabi_base64_decoded_size(output_length: *mut usize, input: *const c_char, input_length: usize) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Calls `qscabi_base64_decoded_size`.

qsc_sys::encoding::qscabi_base64_encode

pub unsafe fn qscabi_base64_encode(output: *mut c_char, output_length: usize, written: *mut usize, input: *const u8, input_length: usize) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Calls `qscabi_base64_encode`.

qsc_sys::encoding::qscabi_base64_decode

pub unsafe fn qscabi_base64_decode(output: *mut u8, output_length: usize, written: *mut usize, input: *const c_char, input_length: usize) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Calls `qscabi_base64_decode`.

qsc_sys::encoding::qscabi_pem_encoded_size

pub unsafe fn qscabi_pem_encoded_size(output_length: *mut usize, label: *const c_char, data_length: usize) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Calls `qscabi_pem_encoded_size`.

qsc_sys::encoding::qscabi_pem_decoded_size

pub unsafe fn qscabi_pem_decoded_size(output_length: *mut usize, input: *const c_char, input_length: usize) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Calls `qscabi_pem_decoded_size`.

qsc_sys::encoding::qscabi_pem_encode

pub unsafe fn qscabi_pem_encode(output: *mut c_char, output_length: usize, written: *mut usize, label: *const c_char, data: *const u8, data_length: usize) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Calls `qscabi_pem_encode`.

qsc_sys::encoding::qscabi_pem_decode

pub unsafe fn qscabi_pem_decode(output: *mut u8, output_length: usize, written: *mut usize, input: *const c_char, input_length: usize) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Calls `qscabi_pem_decode`.

22.3 entropy.rs (10 declared Rust functions)

qsc_sys::entropy::qscabi_entropy_acp_seed_max

pub unsafe fn qscabi_entropy_acp_seed_max() -> Result<usize, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::entropy::qscabi_entropy_csp_seed_max

```
pub unsafe fn qscabi_entropy_csp_seed_max() -> Result<usize, crate::loader::LoadError>
```

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::entropy::qscabi_entropy_rdp_seed_max

```
pub unsafe fn qscabi_entropy_rdp_seed_max() -> Result<usize, crate::loader::LoadError>
```

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::entropy::qscabi_entropy_cjp_seed_max

```
pub unsafe fn qscabi_entropy_cjp_seed_max() -> Result<usize, crate::loader::LoadError>
```

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::entropy::qscabi_entropy_rdp_available

```
pub unsafe fn qscabi_entropy_rdp_available() -> Result<c_int, crate::loader::LoadError>
```

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::entropy::qscabi_entropy_cjp_available

```
pub unsafe fn qscabi_entropy_cjp_available() -> Result<c_int, crate::loader::LoadError>
```

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::entropy::qscabi_entropy_acp_generate

```
pub unsafe fn qscabi_entropy_acp_generate(output: *mut u8, output_length: usize) -> Result<QscAbiStatus, crate::loader::LoadError>
```

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::entropy::qscabi_entropy_csp_generate

```
pub unsafe fn qscabi_entropy_csp_generate(output: *mut u8, output_length: usize) -> Result<QscAbiStatus, crate::loader::LoadError>
```

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::entropy::qscabi_entropy_rdp_generate

```
pub unsafe fn qscabi_entropy_rdp_generate(output: *mut u8, output_length: usize) -> Result<QscAbiStatus, crate::loader::LoadError>
```

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::entropy::qscabi_entropy_cjp_generate

```
pub unsafe fn qscabi_entropy_cjp_generate(output: *mut u8, output_length: usize) -> Result<QscAbiStatus, crate::loader::LoadError>
```

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

22.4 explicit_curve.rs (6 declared Rust functions)

qsc_sys::explicit_curve::available

```
pub fn available() -> bool
```

Source contract: Return whether the selected QSCABI has a named-curve sizing entrypoint.

qsc_sys::explicit_curve::get_sizes

```
pub unsafe fn get_sizes(algorithm: CurveAlgorithm, output: *mut CurveSizes) -> Result<QscAbiStatus, LoadError>
```

Source contract: Call the native curve sizing function.

qsc_sys::explicit_curve::generate

```
pub unsafe fn generate(algorithm: CurveAlgorithm, public: *mut u8, public_length: usize, private: *mut u8, private_length: usize) -> Result<QscAbiStatus, LoadError>
```

Source contract: Generate a selected curve keypair into two caller-owned buffers.

qsc_sys::explicit_curve::generate_seeded

```
pub unsafe fn generate_seeded(algorithm: CurveAlgorithm, public: *mut u8, public_length: usize, private: *mut u8, private_length: usize, seed: *const u8, seed_length: usize) -> Result<QscAbiStatus, LoadError>
```

Source contract: Deterministically generate a curve keypair using caller-supplied seed bytes.

qsc_sys::explicit_curve::public_from_private

```
pub unsafe fn public_from_private(algorithm: CurveAlgorithm, public: *mut u8, public_length: usize, private: *const u8, private_length: usize) -> Result<QscAbiStatus, LoadError>
```

Source contract: Derive a curve public key from the matching private scalar.

qsc_sys::explicit_curve::agree

```
pub unsafe fn agree(algorithm: CurveAlgorithm, secret: *mut u8, secret_length: usize, peer_public: *const u8, peer_length: usize, private: *const u8, private_length: usize) -> Result<QscAbiStatus, LoadError>
```

Source contract: Compute one candidate shared secret from a peer public key and local key.

22.5 explicit_kem.rs (5 declared Rust functions)

qsc_sys::explicit_kem::available

```
pub fn available() -> bool
```

Source contract: Determine whether the named KEM sizing entrypoint is available.

qsc_sys::explicit_kem::get_sizes

```
pub unsafe fn get_sizes(algorithm: KemAlgorithm, sizes: *mut KemSizes) -> Result<QscAbiStatus, LoadError>
```

Source contract: Resolve and invoke `qscabi_kem_get_sizes`.

qsc_sys::explicit_kem::generate

```
pub unsafe fn generate(algorithm: KemAlgorithm, public_key: *mut u8, public_length: usize, private_key: *mut u8, private_length: usize) -> Result<QscAbiStatus, LoadError>
```

Source contract: Resolve and invoke `qscabi_kem_generate` with caller-owned key buffers.

qsc_sys::explicit_kem::encapsulate

pub unsafe fn encapsulate(algorithm: KemAlgorithm, ciphertext: *mut u8, ciphertext_length: usize, secret: *mut u8, secret_length: usize, public_key: *const u8, public_length: usize) -> Result<QscAbiStatus, LoadError>

Source contract: Resolve and invoke `qscabi_kem_encapsulate` with borrowed public input.

qsc_sys::explicit_kem::decapsulate

pub unsafe fn decapsulate(algorithm: KemAlgorithm, secret: *mut u8, secret_length: usize, ciphertext: *const u8, ciphertext_length: usize, private_key: *const u8, private_length: usize) -> Result<QscAbiStatus, LoadError>

Source contract: Resolve and invoke `qscabi_kem_decapsulate` with a borrowed private key.

22.6 explicit_signature.rs (5 declared Rust functions)

qsc_sys::explicit_signature::available

pub fn available() -> bool

Source contract: Report availability of the newer attached-signature sizing symbol.

qsc_sys::explicit_signature::get_sizes

pub unsafe fn get_sizes(algorithm: SignatureAlgorithm, sizes: *mut SignatureSizes) -> Result<QscAbiStatus, LoadError>

Source contract: Call the native attached-signature size query.

qsc_sys::explicit_signature::generate

pub unsafe fn generate(algorithm: SignatureAlgorithm, public: *mut u8, public_length: usize, private: *mut u8, private_length: usize) -> Result<QscAbiStatus, LoadError>

Source contract: Write a freshly generated public/private signing keypair.

qsc_sys::explicit_signature::sign

pub unsafe fn sign(algorithm: SignatureAlgorithm, signed: *mut u8, capacity: usize, written: *mut usize, message: *const u8, message_length: usize, private: *const u8, private_length: usize) -> Result<QscAbiStatus, LoadError>

Source contract: Sign borrowed message bytes into caller-owned attached-message storage.

qsc_sys::explicit_signature::verify

pub unsafe fn verify(algorithm: SignatureAlgorithm, message: *mut u8, capacity: usize, written: *mut usize, signed: *const u8, signed_length: usize, public: *const u8, public_length: usize) -> Result<QscAbiStatus, LoadError>

Source contract: Verify an attached signature and write the recovered message on success.

22.7 explicit_slhdsa.rs (5 declared Rust functions)

qsc_sys::explicit_slhdsa::available

pub fn available() -> bool

Source contract: Return whether the newer SLH-DSA size-query export is present.

qsc_sys::explicit_slhdsa::get_sizes

pub unsafe fn get_sizes(algorithm: SlhDsaAlgorithm, sizes: *mut SlhDsaSizes) -> Result<QscAbiStatus, LoadError>

Source contract: Retrieve native buffer sizes for the selected variant.

qsc_sys::explicit_slhdsa::generate

pub unsafe fn generate(algorithm: SlhDsaAlgorithm, public: *mut u8, public_length: usize, private: *mut u8, private_length: usize) -> Result<QscAbiStatus, LoadError>

Source contract: Generate a new SLH-DSA keypair using native CSP entropy.

qsc_sys::explicit_slhdsa::sign

pub unsafe fn sign(algorithm: SlhDsaAlgorithm, prehash: SlhDsaPrehash, randomized: bool, signature: *mut u8, signature_length: usize, message: *const u8, message_length: usize, context: *const u8, context_length: usize, private: *const u8, private_length: usize) -> Result<QscAbiStatus, LoadError>

Source contract: Sign a nonempty message and optional context as a detached signature.

qsc_sys::explicit_slhdsa::verify

pub unsafe fn verify(algorithm: SlhDsaAlgorithm, prehash: SlhDsaPrehash, signature: *const u8, signature_length: usize, message: *const u8, message_length: usize, context: *const u8, context_length: usize, public: *const u8, public_length: usize) -> Result<QscAbiStatus, LoadError>

Source contract: Verify a detached SLH-DSA signature for the exact prehash, message and context.

22.8 hardware.rs (14 declared Rust functions)

qsc_sys::hardware::hw_is_enabled

pub unsafe fn hw_is_enabled() -> Result<c_int, LoadError>

Source contract: Bind the exact native `qscabi_hw_is_enabled` declaration from `qscabi_hardware.h`.

qsc_sys::hardware::hw_provider_open

pub unsafe fn hw_provider_open(provider: *mut *mut c_void, operations: *const HwBackendOps, context: *mut c_void) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_hw_provider_open` declaration from `qscabi_hardware.h`.

qsc_sys::hardware::hw_provider_capabilities

pub unsafe fn hw_provider_capabilities(provider: *mut c_void, capabilities: *mut HwCapabilities) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_hw_provider_capabilities` declaration from `qscabi_hardware.h`.

qsc_sys::hardware::hw_key_open

pub unsafe fn hw_key_open(key: *mut *mut c_void, provider: *mut c_void, locator: *const c_char, policy: *const HwPolicy) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_hw_key_open` declaration from `qscabi_hardware.h`.

qsc_sys::hardware::hw_key_generate

pub unsafe fn hw_key_generate(key: *mut *mut c_void, provider: *mut c_void, policy: *const HwPolicy) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_hw_key_generate` declaration from `qscabi_hardware.h`.

qsc_sys::hardware::hw_key_get_info

pub unsafe fn hw_key_get_info(key: *const c_void, info: *mut HwKeyInfo) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_hw_key_get_info` declaration from `qscabi_hardware.h`.

qsc_sys::hardware::hw_key_public

pub unsafe fn hw_key_public(key: *mut c_void, output: *mut u8, output_length: *mut usize) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_hw_key_public` declaration from `qscabi_hardware.h`.

qsc_sys::hardware::hw_key_sign

pub unsafe fn hw_key_sign(key: *mut c_void, message: *const u8, message_length: usize, signature: *mut u8, signature_length: *mut usize) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_hw_key_sign` declaration from `qscabi_hardware.h`.

qsc_sys::hardware::hw_key_agree

pub unsafe fn hw_key_agree(key: *mut c_void, peer: *const u8, peer_length: usize, secret: *mut u8, secret_length: *mut usize) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_hw_key_agree` declaration from `qscabi_hardware.h`.

qsc_sys::hardware::hw_key_attest

pub unsafe fn hw_key_attest(key: *mut c_void, challenge: *const u8, challenge_length: usize, evidence: *mut u8, evidence_length: *mut usize) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_hw_key_attest` declaration from `qscabi_hardware.h`.

qsc_sys::hardware::hw_provider_random

pub unsafe fn hw_provider_random(provider: *mut c_void, output: *mut u8, length: usize) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_hw_provider_random` declaration from `qscabi_hardware.h`.

qsc_sys::hardware::hw_provider_invalidate

pub unsafe fn hw_provider_invalidate(provider: *mut c_void) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_hw_provider_invalidate` declaration from `qscabi_hardware.h`.

qsc_sys::hardware::hw_key_release

pub unsafe fn hw_key_release(key: *mut *mut c_void) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_hw_key_release` declaration from `qscabi_hardware.h`.

qsc_sys::hardware::hw_provider_close

pub unsafe fn hw_provider_close(provider: *mut *mut c_void) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_hw_provider_close` declaration from `qscabi_hardware.h`.

22.9 kdf.rs (1 declared Rust functions)

qsc_sys::kdf::qscabi_pbmac1_derive_key

pub unsafe fn qscabi_pbmac1_derive_key(output: *mut u8, output_length: usize, hash: QscAbiPbmac1Hash, password: *const u8, password_length: usize, salt: *const u8, salt_length: usize, iterations: u64, key_length: usize) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

22.10 mac.rs (6 declared Rust functions)

qsc_sys::mac::qscabi_pbmac1_mac_size

pub unsafe fn qscabi_pbmac1_mac_size(hash: QscAbiPbmac1Hash) -> Result<usize, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::mac::qscabi_poly1305_compute

pub unsafe fn qscabi_poly1305_compute(output: *mut u8, output_length: usize, message: *const u8, message_length: usize, key: *const u8, key_length: usize) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::mac::qscabi_poly1305_verify

pub unsafe fn qscabi_poly1305_verify(code: *const u8, code_length: usize, message: *const u8, message_length: usize, key: *const u8, key_length: usize) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::mac::qscabi_qmac_compute

pub unsafe fn qscabi_qmac_compute(output: *mut u8, output_length: usize, message: *const u8, message_length: usize, key: *const u8, key_length: usize, nonce: *const u8, nonce_length: usize, info: *const u8, info_length: usize) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::mac::qscabi_pbmac1_compute

pub unsafe fn qscabi_pbmac1_compute(output: *mut u8, output_length: usize, hash: QscAbiPbmac1Hash, password: *const u8, password_length: usize, salt: *const u8, salt_length: usize, iterations: u64, key_length: usize, message: *const u8, message_length: usize) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::mac::qscabi_pbmac1_verify

pub unsafe fn qscabi_pbmac1_verify(code: *const u8, code_length: usize, hash: QscAbiPbmac1Hash, password: *const u8, password_length: usize, salt: *const u8, salt_length: usize, iterations: u64, key_length: usize, message: *const u8, message_length: usize) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

22.11 memory.rs (2 declared Rust functions)

qsc_sys::memory::qscabi_memory_clear

pub unsafe fn qscabi_memory_clear(data: *mut u8, length: usize,) -> Result<(), crate::loader::LoadError>

Source contract: Calls `qscabi_memory_clear`.

qsc_sys::memory::qscabi_memory_are_equal

pub unsafe fn qscabi_memory_are_equal(left: *const u8, right: *const u8, length: usize,) -> Result<c_int, crate::loader::LoadError>

Source contract: Calls `qscabi_memory_are_equal`.

22.12 status.rs (1 declared Rust functions)

qsc_sys::status::qscabi_status_string

pub unsafe fn qscabi_status_string(status: QscAbiStatus,) -> Result<*const core::ffi::c_char, crate::loader::LoadError>

Source contract: Calls `qscabi_status_string` for the supplied status.

22.13 streaming.rs (18 declared Rust functions)

qsc_sys::streaming::digest_size

pub unsafe fn digest_size(algorithm: c_int) -> Result<usize, LoadError>

Source contract: Bind the exact native `qscabi_digest_size` declaration from `qscabi_stream.h`.

qsc_sys::streaming::digest_create

pub unsafe fn digest_create(algorithm: c_int, state: *mut *mut c_void) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_digest_create` declaration from `qscabi_stream.h`.

qsc_sys::streaming::digest_update

pub unsafe fn digest_update(state: *mut c_void, data: *const u8, length: usize) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_digest_update` declaration from `qscabi_stream.h`.

qsc_sys::streaming::digest_finalize

pub unsafe fn digest_finalize(state: *mut c_void, digest: *mut u8, digest_length: usize) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_digest_finalize` declaration from `qscabi_stream.h`.

qsc_sys::streaming::digest_destroy

pub unsafe fn digest_destroy(state: *mut c_void) -> Result<(), LoadError>

Source contract: Bind the exact native `qscabi_digest_destroy` declaration from `qscabi_stream.h`.

qsc_sys::streaming::xof_create

pub unsafe fn xof_create(algorithm: c_int, state: *mut *mut c_void) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_xof_create` declaration from `qscabi_stream.h`.

qsc_sys::streaming::xof_absorb

pub unsafe fn xof_absorb(state: *mut c_void, data: *const u8, length: usize) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_xof_absorb` declaration from `qscabi_stream.h`.

qsc_sys::streaming::xof_finalize

pub unsafe fn xof_finalize(state: *mut c_void) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_xof_finalize` declaration from `qscabi_stream.h`.

qsc_sys::streaming::xof_squeeze

```
pub unsafe fn xof_squeeze(state: *mut c_void, output: *mut u8, length: usize) -> Result<QscAbiStatus, LoadError>
```

Source contract: Bind the exact native `qscabi_xof_squeeze` declaration from `qscabi_stream.h`.

qsc_sys::streaming::xof_destroy

```
pub unsafe fn xof_destroy(state: *mut c_void) -> Result<(), LoadError>
```

Source contract: Bind the exact native `qscabi_xof_destroy` declaration from `qscabi_stream.h`.

qsc_sys::streaming::cshake_compute

```
pub unsafe fn cshake_compute(algorithm: c_int, key: *const u8, key_length: usize, name: *const u8, name_length: usize, custom: *const u8, custom_length: usize, output: *mut u8, output_length: usize) -> Result<QscAbiStatus, LoadError>
```

Source contract: Bind the exact native `qscabi_cshake_compute` declaration from `qscabi_stream.h`.

qsc_sys::streaming::stream_mac_create

```
pub unsafe fn stream_mac_create(algorithm: c_int, key: *const u8, key_length: usize, custom: *const u8, custom_length: usize, state: *mut *mut c_void) -> Result<QscAbiStatus, LoadError>
```

Source contract: Bind the exact native `qscabi_stream_mac_create` declaration from `qscabi_stream.h`.

qsc_sys::streaming::stream_mac_update

```
pub unsafe fn stream_mac_update(state: *mut c_void, data: *const u8, length: usize) -> Result<QscAbiStatus, LoadError>
```

Source contract: Bind the exact native `qscabi_stream_mac_update` declaration from `qscabi_stream.h`.

qsc_sys::streaming::stream_mac_finalize

```
pub unsafe fn stream_mac_finalize(state: *mut c_void, output: *mut u8, length: usize) -> Result<QscAbiStatus, LoadError>
```

Source contract: Bind the exact native `qscabi_stream_mac_finalize` declaration from `qscabi_stream.h`.

qsc_sys::streaming::stream_mac_destroy

```
pub unsafe fn stream_mac_destroy(state: *mut c_void) -> Result<(), LoadError>
```

Source contract: Bind the exact native `qscabi_stream_mac_destroy` declaration from `qscabi_stream.h`.

qsc_sys::streaming::ctr_create

```
pub unsafe fn ctr_create(algorithm: c_int, key: *const u8, key_length: usize, nonce: *const u8, nonce_length: usize, state: *mut *mut c_void) -> Result<QscAbiStatus, LoadError>
```

Source contract: Bind the exact native `qscabi_ctr_create` declaration from `qscabi_stream.h`.

qsc_sys::streaming::ctr_transform

```
pub unsafe fn ctr_transform(state: *mut c_void, input: *const u8, length: usize, output: *mut u8, output_length: usize) -> Result<QscAbiStatus, LoadError>
```

Source contract: Bind the exact native `qscabi_ctr_transform` declaration from `qscabi_stream.h`.

qsc_sys::streaming::ctr_destroy

```
pub unsafe fn ctr_destroy(state: *mut c_void) -> Result<(), LoadError>
```

Source contract: Bind the exact native `qscabi_ctr_destroy` declaration from `qscabi_stream.h`.

22.14 symmetric.rs (8 declared Rust functions)

qsc_sys::symmetric::qscabi_rcs_key_size

pub unsafe fn qscabi_rcs_key_size(mode: QscAbiRcsMode) -> Result<usize, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::symmetric::qscabi_rcs_tag_size

pub unsafe fn qscabi_rcs_tag_size(mode: QscAbiRcsMode) -> Result<usize, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::symmetric::qscabi_aes256_ctr_transform

pub unsafe fn qscabi_aes256_ctr_transform(output: *mut u8, output_length: usize, input: *const u8, input_length: usize, key: *const u8, key_length: usize, nonce: *const u8, nonce_length: usize, counter_mode: QscAbiAesCtrCounter,) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::symmetric::qscabi_chacha20_transform

pub unsafe fn qscabi_chacha20_transform(output: *mut u8, output_length: usize, input: *const u8, input_length: usize, key: *const u8, key_length: usize, nonce: *const u8, nonce_length: usize,) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::symmetric::qscabi_chacha20_poly1305_encrypt

pub unsafe fn qscabi_chacha20_poly1305_encrypt(output: *mut u8, output_length: usize, written: *mut usize, plaintext: *const u8, plaintext_length: usize, key: *const u8, key_length: usize, nonce: *const u8, nonce_length: usize, aad: *const u8, aad_length: usize,) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::symmetric::qscabi_chacha20_poly1305_decrypt

pub unsafe fn qscabi_chacha20_poly1305_decrypt(output: *mut u8, output_length: usize, written: *mut usize, ciphertext: *const u8, ciphertext_length: usize, key: *const u8, key_length: usize, nonce: *const u8, nonce_length: usize, aad: *const u8, aad_length: usize,) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::symmetric::qscabi_rcs_encrypt

pub unsafe fn qscabi_rcs_encrypt(output: *mut u8, output_length: usize, written: *mut usize, mode: QscAbiRcsMode, plaintext: *const u8, plaintext_length: usize, key: *const u8, key_length: usize, nonce: *const u8, nonce_length: usize, info: *const u8, info_length: usize, aad: *const u8, aad_length: usize,) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::symmetric::qscabi_rcs_decrypt

pub unsafe fn qscabi_rcs_decrypt(output: *mut u8, output_length: usize, written: *mut usize, mode: QscAbiRcsMode, ciphertext: *const u8, ciphertext_length: usize, key: *const u8, key_length: usize, nonce: *const u8, nonce_length: usize, info: *const u8, info_length: usize, aad: *const u8, aad_length: usize,) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

22.15 tls.rs (32 declared Rust functions)

qsc_sys::tls::qscabi_tls_status_string

pub unsafe fn qscabi_tls_status_string(status: QscAbiStatus) -> Result<*const c_char, crate::loader::LoadError>

Source contract: Gets a TLS-related status string.

qsc_sys::tls::qscabi_tls_socket_options_default

pub unsafe fn qscabi_tls_socket_options_default(options: *mut QscAbiTlsSocketOptions) -> Result<(), crate::loader::LoadError>

Source contract: Fills socket options with native QSCABI defaults.

qsc_sys::tls::qscabi_tls_ticket_policy_default

pub unsafe fn qscabi_tls_ticket_policy_default(policy: *mut QscAbiTlsTicketPolicy) -> Result<(), crate::loader::LoadError>

Source contract: Fills ticket policy with native QSCABI defaults.

qsc_sys::tls::qscabi_tls_context_set_policy

pub unsafe fn qscabi_tls_context_set_policy(context: *mut c_void, policy: QscAbiTlsPolicy) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_context_set_socket_options

pub unsafe fn qscabi_tls_context_set_socket_options(context: *mut c_void, options: *const QscAbiTlsSocketOptions) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_context_set_alpn_protocols

pub unsafe fn qscabi_tls_context_set_alpn_protocols(context: *mut c_void, protocols: *const *const c_char, protocol_count: usize, required: c_int) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_context_clear_alpn_protocols

pub unsafe fn qscabi_tls_context_clear_alpn_protocols(context: *mut c_void) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_context_set_sni_required

pub unsafe fn qscabi_tls_context_set_sni_required(context: *mut c_void, required: c_int) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_context_set_client_auth

pub unsafe fn qscabi_tls_context_set_client_auth(context: *mut c_void, request_client_auth: c_int, require_client_auth: c_int) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_context_set_ticket_policy

pub unsafe fn qscabi_tls_context_set_ticket_policy(context: *mut c_void, policy: *const QscAbiTlsTicketPolicy) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_context_load_trust_anchor_pem

pub unsafe fn qscabi_tls_context_load_trust_anchor_pem(context: *mut c_void, pem: *const c_char, pem_length: usize, self_signed: c_int) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_context_load_server_identity_pem

pub unsafe fn qscabi_tls_context_load_server_identity_pem(context: *mut c_void, chain_pem: *const c_char, chain_pem_length: usize, private_key_pem: *const c_char, private_key_pem_length: usize, signature_scheme: QscAbiTlsSignatureScheme) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_client_connect_host

pub unsafe fn qscabi_tls_client_connect_host(connection: *mut c_void, context: *const c_void, hostname: *const c_char, service: *const c_char) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_connection_send

pub unsafe fn qscabi_tls_connection_send(connection: *mut c_void, input: *const u8, input_length: usize, written: *mut usize) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_connection_receive

pub unsafe fn qscabi_tls_connection_receive(connection: *mut c_void, output: *mut u8, output_length: usize, read_length: *mut usize) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_connection_send_frame

pub unsafe fn qscabi_tls_connection_send_frame(connection: *mut c_void, input: *const u8, input_length: usize) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_connection_receive_frame

pub unsafe fn qscabi_tls_connection_receive_frame(connection: *mut c_void, output: *mut u8, output_length: usize, read_length: *mut usize) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_connection_shutdown

pub unsafe fn qscabi_tls_connection_shutdown(connection: *mut c_void) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_connection_cancel

pub unsafe fn qscabi_tls_connection_cancel(connection: *mut c_void) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_connection_is_connected

pub unsafe fn qscabi_tls_connection_is_connected(connection: *const c_void) -> Result<c_int, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_connection_is_handshake_complete

pub unsafe fn qscabi_tls_connection_is_handshake_complete(connection: *const c_void) -> Result<c_int, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_connection_get_selected_alpn

pub unsafe fn qscabi_tls_connection_get_selected_alpn(connection: *const c_void, output: *mut c_char, output_length: usize, written: *mut usize) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_connection_get_peer_info

pub unsafe fn qscabi_tls_connection_get_peer_info(connection: *const c_void, peer_info: *mut QscAbiTlsPeerInfo) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_listener_set_socket_options

pub unsafe fn qscabi_tls_listener_set_socket_options(listener: *mut c_void, options: *const QscAbiTlsSocketOptions) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_listener_bind

pub unsafe fn qscabi_tls_listener_bind(listener: *mut c_void, context: *const c_void, address: *const c_char, port: u16, family: QscAbiTlsAddressFamily) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_listener_accept

pub unsafe fn qscabi_tls_listener_accept(listener: *mut c_void, connection: *mut c_void) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_listener_close

pub unsafe fn qscabi_tls_listener_close(listener: *mut c_void) -> Result<(), crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_server_configure

pub unsafe fn qscabi_tls_server_configure(server: *mut c_void, context: *const c_void, address: *const c_char, port: u16, family: QscAbiTlsAddressFamily) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_server_set_max_clients

pub unsafe fn qscabi_tls_server_set_max_clients(server: *mut c_void, max_clients: usize) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_server_start

pub unsafe fn qscabi_tls_server_start(server: *mut c_void) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_server_start_concurrent

pub unsafe fn qscabi_tls_server_start_concurrent(server: *mut c_void) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

qsc_sys::tls::qscabi_tls_server_stop

pub unsafe fn qscabi_tls_server_stop(server: *mut c_void) -> Result<(), crate::loader::LoadError>

Source contract: Raw native binding; caller must validate all pointer lengths, status results and loader failures.

22.16 tls010.rs (9 declared Rust functions)

qsc_sys::tls010::tls_context_set_cipher_suites

pub unsafe fn tls_context_set_cipher_suites(context: *mut c_void, suites: *const u16, count: usize) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_tls_context_set_cipher_suites` declaration from `qscabi_tls.h`.

qsc_sys::tls010::tls_context_set_named_groups

pub unsafe fn tls_context_set_named_groups(context: *mut c_void, groups: *const u16, count: usize) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_tls_context_set_named_groups` declaration from `qscabi_tls.h`.

qsc_sys::tls010::tls_context_set_signature_ids

pub unsafe fn tls_context_set_signature_ids(context: *mut c_void, schemes: *const u16, count: usize) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_tls_context_set_signature_ids` declaration from `qscabi_tls.h`.

qsc_sys::tls010::tls_context_load_trust_anchor_file

pub unsafe fn tls_context_load_trust_anchor_file(context: *mut c_void, path: *const c_char, self_signed: c_int) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_tls_context_load_trust_anchor_file` declaration from `qscabi_tls.h`.

qsc_sys::tls010::tls_context_load_trust_anchor_bundle_file

pub unsafe fn tls_context_load_trust_anchor_bundle_file(context: *mut c_void, path: *const c_char, self_signed: c_int) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_tls_context_load_trust_anchor_bundle_file` declaration from `qscabi_tls.h`.

qsc_sys::tls010::tls_context_load_crl_file

pub unsafe fn tls_context_load_crl_file(context: *mut c_void, path: *const c_char) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_tls_context_load_crl_file` declaration from `qscabi_tls.h`.

qsc_sys::tls010::tls_context_load_server_identity_files

pub unsafe fn tls_context_load_server_identity_files(context: *mut c_void, chain_path: *const c_char, key_path: *const c_char, scheme: c_int) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_tls_context_load_server_identity_files` declaration from `qscabi_tls.h`.

qsc_sys::tls010::tls_context_load_client_identity_files

pub unsafe fn tls_context_load_client_identity_files(context: *mut c_void, chain_path: *const c_char, key_path: *const c_char, scheme: c_int) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_tls_context_load_client_identity_files` declaration from `qscabi_tls.h`.

qsc_sys::tls010::tls_context_add_server_identity_files

pub unsafe fn tls_context_add_server_identity_files(context: *mut c_void, hostname: *const c_char, chain_path: *const c_char, key_path: *const c_char, scheme: c_int) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_tls_context_add_server_identity_files` declaration from `qscabi_tls.h`.

22.17 version.rs (8 declared Rust functions)

qsc_sys::version::qscabi_version_major

pub unsafe fn qscabi_version_major() -> Result<u32, crate::loader::LoadError>

Source contract: Calls `qscabi_version_major`.

qsc_sys::version::qscabi_version_minor

pub unsafe fn qscabi_version_minor() -> Result<u32, crate::loader::LoadError>

Source contract: Calls `qscabi_version_minor`.

qsc_sys::version::qscabi_version_patch

pub unsafe fn qscabi_version_patch() -> Result<u32, crate::loader::LoadError>

Source contract: Calls `qscabi_version_patch`.

qsc_sys::version::qscabi_version_string

pub unsafe fn qscabi_version_string() -> Result<*const c_char, crate::loader::LoadError>

Source contract: Calls `qscabi_version_string`.

qsc_sys::version::qscabi_qsc_version_string

pub unsafe fn qscabi_qsc_version_string() -> Result<*const c_char, crate::loader::LoadError>

Source contract: Calls `qscabi_qsc_version_string`.

qsc_sys::version::qscabi_build_platform

```
pub unsafe fn qscabi_build_platform() -> Result<*const c_char, crate::loader::LoadError>
```

Source contract: Calls `qscabi_build_platform`.

qsc_sys::version::qscabi_build_compiler

```
pub unsafe fn qscabi_build_compiler() -> Result<*const c_char, crate::loader::LoadError>
```

Source contract: Calls `qscabi_build_compiler`.

qsc_sys::version::qscabi_enabled_features

```
pub unsafe fn qscabi_enabled_features() -> Result<*const c_char, crate::loader::LoadError>
```

Source contract: Calls `qscabi_enabled_features`.

22.18 x509.rs (16 declared Rust functions)

qsc_sys::x509::qscabi_x509_certificate_create

```
pub unsafe fn qscabi_x509_certificate_create(certificate: *mut *mut c_void) -> Result<QscAbiStatus, crate::loader::LoadError>
```

Source contract: Allocates an opaque X.509 certificate handle.

qsc_sys::x509::qscabi_x509_certificate_destroy

```
pub unsafe fn qscabi_x509_certificate_destroy(certificate: *mut c_void) -> Result<(), crate::loader::LoadError>
```

Source contract: Releases an opaque X.509 certificate handle.

qsc_sys::x509::qscabi_x509_certificate_subject_dns_name_count

```
pub unsafe fn qscabi_x509_certificate_subject_dns_name_count(certificate: *const c_void) -> Result<usize, crate::loader::LoadError>
```

Source contract: Returns the number of DNS subject-alternative-name entries in a certificate.

qsc_sys::x509::qscabi_x509_certificate_subject_dns_name

```
pub unsafe fn qscabi_x509_certificate_subject_dns_name( certificate: *const c_void, index: usize, output: *mut c_char, output_length: usize, written: *mut usize, ) -> Result<QscAbiStatus, crate::loader::LoadError>
```

Source contract: Reads a DNS subject-alternative-name entry by index.

qsc_sys::x509::qscabi_x509_certificate_match_hostname

```
pub unsafe fn qscabi_x509_certificate_match_hostname( certificate: *const c_void, hostname: *const c_char, ) -> Result<QscAbiStatus, crate::loader::LoadError>
```

Source contract: Checks whether a certificate matches a hostname.

qsc_sys::x509::qscabi_x509_certificate_check_role

```
pub unsafe fn qscabi_x509_certificate_check_role( certificate: *const c_void, role: QscAbiX509CertificateRole, hostname: *const c_char, ) -> Result<QscAbiStatus, crate::loader::LoadError>
```

Source contract: Checks whether a certificate satisfies a requested role.

qsc_sys::x509::qscabi_x509_chain_create

```
pub unsafe fn qscabi_x509_chain_create(chain: *mut *mut c_void, certificate_capacity: usize) -> Result<QscAbiStatus, crate::loader::LoadError>
```

Source contract: Allocates an opaque certificate-chain handle.

qsc_sys::x509::qscabi_x509_chain_destroy

pub unsafe fn qscabi_x509_chain_destroy(chain: *mut c_void) -> Result<(), crate::loader::LoadError>

Source contract: Releases an opaque certificate-chain handle.

qsc_sys::x509::qscabi_x509_chain_load_memory

pub unsafe fn qscabi_x509_chain_load_memory(chain: *mut c_void, data: *const u8, data_length: usize,) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Loads an encoded certificate chain from memory.

qsc_sys::x509::qscabi_x509_chain_count

pub unsafe fn qscabi_x509_chain_count(chain: *const c_void) -> Result<usize, crate::loader::LoadError>

Source contract: Returns the number of certificates currently loaded into a chain.

qsc_sys::x509::qscabi_x509_store_create

pub unsafe fn qscabi_x509_store_create(store: *mut *mut c_void, anchor_capacity: usize) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Allocates an opaque X.509 trust-store handle.

qsc_sys::x509::qscabi_x509_store_destroy

pub unsafe fn qscabi_x509_store_destroy(store: *mut c_void) -> Result<(), crate::loader::LoadError>

Source contract: Releases an opaque X.509 trust-store handle.

qsc_sys::x509::qscabi_x509_store_add_anchor_memory

pub unsafe fn qscabi_x509_store_add_anchor_memory(store: *mut c_void, data: *const u8, data_length: usize, self_signed: c_int,) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Adds a trust anchor from a memory buffer.

qsc_sys::x509::qscabi_x509_store_count

pub unsafe fn qscabi_x509_store_count(store: *const c_void) -> Result<usize, crate::loader::LoadError>

Source contract: Returns the number of trust anchors currently loaded into a store.

qsc_sys::x509::qscabi_x509_verify_peer_certificates

pub unsafe fn qscabi_x509_verify_peer_certificates(chain: *const c_void, store: *const c_void, preset: QscAbiX509ProfilePreset, hostname: *const c_char,) -> Result<QscAbiStatus, crate::loader::LoadError>

Source contract: Verifies a peer certificate chain against a trust store.

qsc_sys::x509::qscabi_x509_status_string

pub unsafe fn qscabi_x509_status_string(status: QscAbiStatus) -> Result<*const c_char, crate::loader::LoadError>

Source contract: Returns the native X.509 status string for an ABI status.

22.19 x509010.rs (15 declared Rust functions)

qsc_sys::x509010::x509_csr_create

pub unsafe fn x509_csr_create(csr: *mut *mut c_void) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_x509_csr_create` declaration from `qscabi_x509.h`.

qsc_sys::x509010::x509_csr_destroy

pub unsafe fn x509_csr_destroy(csr: *mut c_void) -> Result<(), LoadError>

Source contract: Bind the exact native `qscabi_x509_csr_destroy` declaration from `qscabi_x509.h`.

qsc_sys::x509010::x509_csr_init_p256

pub unsafe fn x509_csr_init_p256(csr: *mut c_void, common_name: *const c_char, public_key: *const u8, public_key_length: usize) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_x509_csr_init_p256` declaration from `qscabi_x509.h`.

qsc_sys::x509010::x509_csr_add_dns

pub unsafe fn x509_csr_add_dns(csr: *mut c_void, dns: *const c_char) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_x509_csr_add_dns` declaration from `qscabi_x509.h`.

qsc_sys::x509010::x509_csr_sign_der

pub unsafe fn x509_csr_sign_der(csr: *const c_void, signer: X509SignCallback, context: *mut c_void, output: *mut u8, output_length: *mut usize) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_x509_csr_sign_der` declaration from `qscabi_x509.h`.

qsc_sys::x509010::x509_csr_load

pub unsafe fn x509_csr_load(csr: *mut c_void, data: *const u8, length: usize) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_x509_csr_load` declaration from `qscabi_x509.h`.

qsc_sys::x509010::x509_csr_verify

pub unsafe fn x509_csr_verify(csr: *const c_void) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_x509_csr_verify` declaration from `qscabi_x509.h`.

qsc_sys::x509010::x509_issue_tls_server_p256

pub unsafe fn x509_issue_tls_server_p256(csr: *const c_void, issuer: *const c_void, serial: *const u8, serial_length: usize, not_before: *const X509Time, not_after: *const X509Time, signer: X509SignCallback, context: *mut c_void, output: *mut u8, output_length: *mut usize) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_x509_issue_tls_server_p256` declaration from `qscabi_x509.h`.

qsc_sys::x509010::x509_crl_builder_create

pub unsafe fn x509_crl_builder_create(builder: *mut *mut c_void, issuer: *const c_void, this_update: *const X509Time, next_update: *const X509Time) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_x509_crl_builder_create` declaration from `qscabi_x509.h`.

qsc_sys::x509010::x509_crl_builder_destroy

pub unsafe fn x509_crl_builder_destroy(builder: *mut c_void) -> Result<(), LoadError>

Source contract: Bind the exact native `qscabi_x509_crl_builder_destroy` declaration from `qscabi_x509.h`.

qsc_sys::x509010::x509_crl_set_number

pub unsafe fn x509_crl_set_number(builder: *mut c_void, value: *const u8, length: usize) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_x509_crl_set_number` declaration from `qscabi_x509.h`.

qsc_sys::x509010::x509_crl_add_revoked

pub unsafe fn x509_crl_add_revoked(builder: *mut c_void, serial: *const u8, serial_length: usize, revoked_at: *const X509Time) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_x509_crl_add_revoked` declaration from `qscabi_x509.h`.

qsc_sys::x509010::x509_crl_sign_der

pub unsafe fn x509_crl_sign_der(builder: *const c_void, issuer: *const c_void, signer: X509SignCallback, context: *mut c_void, output: *mut u8, output_length: *mut usize) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_x509_crl_sign_der` declaration from `qscabi_x509.h`.

qsc_sys::x509010::x509_crl_verify_der

pub unsafe fn x509_crl_verify_der(der: *const u8, length: usize, issuer: *const c_void, verification_time: *const X509Time) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_x509_crl_verify_der` declaration from `qscabi_x509.h`.

qsc_sys::x509010::x509_store_add_crl_memory

pub unsafe fn x509_store_add_crl_memory(store: *mut c_void, der: *const u8, length: usize) -> Result<QscAbiStatus, LoadError>

Source contract: Bind the exact native `qscabi_x509_store_add_crl_memory` declaration from `qscabi_x509.h`.

22.99 The eager function table and additive signature freeze

qsc-sys/src/loader.rs contains the major eager native dispatch structure, while source modules such as abi010.rs, x509010.rs, tls010.rs, streaming.rs and hardware.rs add explicit runtime-resolved entry points. The frozen JSON and public symbol text are the machine-readable sources for exact C symbol identity. Do not alter these signatures or ABI tables in documentation-only revisions.

23: Implementor Acceptance Checklist

- Deploy the restored complete workspace and identify the exact rustc/Cargo versions, targets and native ABI build.
- Check qsc::native_available, ABI major/minor and all 329 required native exports.
- Keep qsc (safe) public APIs separate from qsc-sys unsafe implementation boundaries.
- Map usize, C integer enums, output lengths, null pointers and raw native callbacks exactly.
- Use safe Vec and SecretBytes ownership, and do not duplicate non-cloneable native handles.
- Reject short buffers, unsupported parameter sets, authentication failures and stale contexts.
- Never reuse AEAD nonces, one-time MAC keys or unauthenticated CTR records without additional integrity.

- Keep named TLS/PKI trust policy independent of successful parsing, signatures and KEM shared secrets.
- Document hardware callback lifetime, device thread policy, non-exportability and fallback requirements.
- Run the staged tests and independent release qualification, recording actual commands and results.
- Verify Rust crate version and QSCABI version separately, and record native CPU/intrinsics profile.
- Publish only authorized QRCS-CASL artifacts and do not modify protected external QSC sources.

Annex A: Public Rust Types and Selectors

Module source	Public type(s)
asymmetric.rs	AsymmetricKeyPair
explicit_slhdsa.rs	SlhDsaKeyPair
hardware.rs	HardwareProvider, HardwareKey
issuance.rs	Csr, CrlBuilder
kem.rs	KemKeyPair, KemEncapsulation
mac.rs	Pbmac1Hash
secret.rs	SecretBytes
signature.rs	SignatureKeyPair
status.rs	QscStatus
streaming.rs	DigestAlgorithm, XofAlgorithm, MacAlgorithm, CtrAlgorithm, Digest, Xof, Mac, Ctr
symmetric.rs	AesCtrCounter, RcsMode
tls.rs	TlsPolicy, SignatureScheme, AddressFamily, SocketOptions, TicketPolicy, PeerInfo, TlsContext, TlsConnection, TlsListener, TlsServer
version.rs	VersionInfo, NativeBuildInfo
x509.rs	CertificateRole, ProfilePreset, Certificate, Chain, TrustStore

Table A.1 Public Rust data types in qsc/src

Public type definitions specify ownership, enum discriminants and native structure conversion where applicable. Some type re-exports originate in qsc-sys and are not reproduced as new definitions in qsc. Native FFI structure field layout and callback signatures must be taken from the low-level module source and corresponding QSCABI header, not inferred from Debug representations or Rust type names.

Annex B: Native ABI and Source Directory Index

Repository path	Purpose
qsc/src	Safe customer-facing Rust crate; 27 module source files.
qsc-sys/src	FFI definitions, dynamic library loader and 0.10 manifest/signature files.
qsc/tests	15 stage-named integration test source files with source-owned fixture.
qsc_examples/src/examples	Seven stage-grouped Rust examples and command runner.
qsc_tests/src	Standalone test executable and options.
QSCSDKRS	Visual Studio NMake project for invoking Cargo scripts.
scripts	ABI contract checker and Windows build/test launcher.
target/debug	Historical compiled/intermediate artifacts included in ZIP; not treated as new results.

Table B.1 Repository source structure

Category	Count	Meaning
Public native symbols	329	Frozen QSCABI 0.10 names in abi010_public_symbols.txt.
Eager native dispatch slots	252	Frozen Rust extern C native pointer fields.
Additional bound functions	77	17 legacy and 60 additive functions defined through optional paths.
Safe Rust function entries	378	Source-declared and macro-generated public functions in qsc crate.
Low-level Rust wrapper declarations	177	Direct low-level pub fn declarations; loader and eager table separate.
Member Cargo.toml files	4	qsc, qsc-sys, qsc_examples and qsc_tests; root workspace file absent.

Table B.2 Source audit figures

Annex C: Testing and Example Inventory

Test source	#[test] cases	Test themes / leading cases
stage10_stream_hash_xof_tests.rs	6	stream_digest_xof_enums_match_header, stream_cshake_length_guard_rejects_zero_or_excessive, stream_sha256_abc_known_answer_if_native_available
stage11_stream_mac_ctr_tests.rs	5	stream_mac_ctr_selectors_match_native, stream_key_and_nonce_length_guard, stream_hmac_sha256_rfc4231_if_native_available
stage12_tls_policy_files_tests.rs	4	tls_010_empty_preference_arrays_rejected_if_native_available, tls_010_policy_setters_if_native_available, tls_010_file_load_rejects_missing_path_if_native_available
stage13_x509_issuance_tests.rs	5	x509_010_time_native_24_byte_layout, x509_010_calendar_validation_rejects_bad_dates, x509_010_csr_creation_and_unsigned_verification_if_native_available
stage14_hardware_provider_tests.rs	3	hardware_010_struct_layouts_match_header, hardware_010_policy_denies_software_and_private_export, hardware_010_mock_provider_lifecycle_if_native_available
stage15_abi_export_contract_tests.rs	3	abi_010_manifest_has_329_unique_symbols, abi_010_all_329_exports_if_native_available, abi_010_strict_full_contract_if_native_available
stage1_core_tests.rs	5	require_native, skip_without_native, status_labels_are_stable
stage2_core_tests.rs	7	require_native, skip_without_native, encoding_round_trips_if_native_available
stage3_symmetric_tests.rs	6	require_native, skip_without_native, symmetric_sizes_are_reported_if_native_available
stage4_asymmetric_tests.rs	5	require_native, skip_without_native, pqc_kem_round_trips_if_native_available
stage5_x509_tests.rs	5	require_native, skip_without_native, x509_load_pem_reads_identity_fields_if_native_available

Test source	#[test] cases	Test themes / leading cases
stage6_tls_tests.rs	5	native_ready, tls_socket_and_ticket_defaults_are_available, tls_context_policy_and_alpn_lifecycle
stage7_explicit_kem_tests.rs	8	explicit_kem_algorithm_ids_match_native_header, explicit_kem_sizes_have_correct_c_layout, explicit_kem_secret_buffer_is_mutable_and_owned
stage8_explicit_slhdsa_tests.rs	4	explicit_slhdsa_algorithm_ids_match_header, explicit_slhdsa_prehash_ids_match_header, explicit_slhdsa_sizes_have_native_layout
stage9_abi_capability_tests.rs	3	abi_010_capability_bits_match_c_header, abi_010_version_compatibility_if_native_available, abi_010_native_build_profile_if_native_available

Table C.1 Rust integration suite inventory

One-to-one native cryptographic qualification is not guaranteed by this source inventory. Tests that skip on missing native runtime must be re-run in strict/native-required mode before claiming production readiness. Every failure should identify its stage, case, expected status and observed value.